Limbaje formale si automate

Note de curs

MAȘINA TURING

Mașina Turing reprezintă formalizarea ideii de mașină de calcul. În general, atunci când vorbim despre modelul abstract al unei mașini de calcul ne referim la un automat, un dispozitiv cu un număr finit de stări, care operează asupra unui șir de simboluri, pe baza unei mulțimi finite de reguli. Există diferite tipuri de automate, care pot fi clasificate, în funcție de complexitatea lor structurală și comportamentală, în automate finite, automate cu stivă, mașini Turing.

Mașinile Turing formează o clasă maximală și stabilă de automate în termenii operațiilor pe care le pot efectua. Ele au fost proiectate pentru a satisface simultan trei criterii:

- să respecte definiția unui automat;
- să fie ușor de descris, formalizat, și raționat;
- să fie cât mai generale, în termenii operațiilor efectuate.

1.1. Definirea mașinii Turing

O mașină Turing constă dintr-un dispozitiv de control finit și o bandă de intrare:

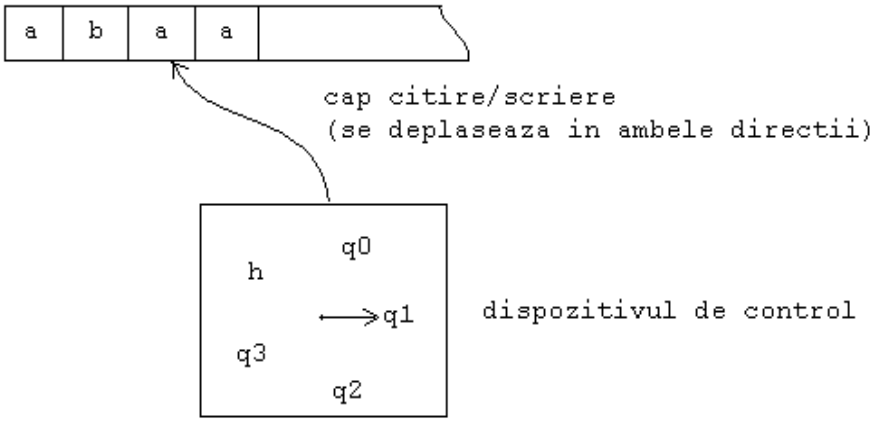


Figura 1-1

Comunicarea între cele două componente este făcută printr-un cap care poate să citească / scrie simboluri pe bandă sau să se deplaseze o poziție la stânga sau la dreapta. Dispozitivul de control acționează în pași discreți. La fiecare pas execută două acțiuni în funcție de starea curentă și de simbolul de pe bandă indicat de capul de citire / scriere, respectiv:

1. pune dispozitivul de control într-o nouă stare, și
2. fie
   a. scrie un simbol pe bandă în poziția curentă, înlocuindu-l pe cel existent
   b. mută capul de citire / scriere o poziție la stânga sau la dreapta

Banda de intrare este limitată la stânga, dar se extinde la dreapta indefinit. Considerăm că banda conține în pozițiile libere simbolul $\#$. Cum mașina poate muta capul doar câte o poziție, după un număr finit de operații numai un număr finit de poziții pot fi accesate. Dacă mașina încearcă să deplaseze capul la stânga capătului benzii, ea încetează să funcționeze.

Pentru că o mașină Turing poate scrie pe bandă, ea poate lăsa un răspuns la sfârșitul operațiilor, de aceea nu trebuie prevăzute stări finale (ca în cazul automatelor finite sau a celor cu stivă).

Există o stare specială de $halt$ (notată h) care este utilizată pentru a indica sfârșitul operațiilor, dar nu oferă informații asupra succesului sau insuccesului operațiilor.

Definiţia 1.1

O mașină Turing este un tuplu (K, Σ, δ, s), unde:

K este mulțimea finită de stări care nu conține starea h;

Σ este un alfabet care conține simbolul $\#$, dar nu conține simbolurile L, R;

$s \in K$ este starea inițială;

δ este funcția de tranziție, $\delta: K \times \Sigma \rightarrow (K \cup \{h\}) \times (\Sigma \cup \{L, R\})$.

Dacă $q \in K$, $a \in \Sigma$, și $\delta(q, a) = (p, b)$ atunci M, dacă este în starea q și scanează simbolul a, intră în starea p. Dacă $b \in \Sigma$ atunci scrie b în locul lui a. Dacă b este L sau R mută capul în direcția b. Cum δ este o

funcție, mașina M este deterministă și se va opri când intră în starea de *halt* sau încearcă să depășească capătul stânga al benzii de intrare.

Exemplu 1.1

Să construim mașina Turing care primind ca intrare un șir de simboluri *a*, cu capul poziționat pe primul *a* din dreapta, se deplasează spre stânga înlocuind *a* cu #. Mașina se oprește pe primul # care delimitează stânga șirul. Fie $M = (K, \Sigma, \delta, s)$, unde

$K = \{q_0, q_1\}$

$\Sigma = \{a, \#\}$

$s = q_0$

funcția de tranziție δ este definită prin tabelul următor:

q	σ	$\delta(q, \sigma)$
q_0	a	$(q_1, \#)$
q_0	#	$(h, \#)$
q_1	a	(q_0, a)
q_1	#	(q_0, L)

M pornește în starea inițială q_0, deplasează capul spre stânga schimbând simbolurile *a* cu # pe măsură ce înaintează, până când găsește un simbol # și se oprește.

Să presupunem că șirul de intrare este format din trei simboluri *a*, primul (din stânga) fiind precedat de #. Mașina pornește cu capul pe primul simbol *a* din dreapta, trece din starea q_0 în q_1 și invers de trei ori schimbând *a* cu # și deplasând capul spre stânga. Când ajunge în starea q_0 cu # pe bandă se oprește. Practic starea q_0 este utilizată pentru a înlocui simbolul *a* de pe bandă cu #, iar starea q_1 pentru a deplasa capul de citire o poziție la stânga. A treia linie din tabel descrie o tranziție care practic nu poate avea loc, dar a fost introdusă pentru că δ este o funcție totală pe domeniul $K \times \Sigma$, și prin urmare trebuie definită pe tot domeniul.

Exemplu 1.2

Să construim mașina Turing care primind un șir de simboluri a la intrare cu capul poziționat pe primul a din stânga șirului, se deplasează la dreapta pâna la simbolul #. Fie $M = (K, \Sigma, \delta, s)$, unde

$K = \{q_0\}$

$\Sigma = \{a, \#\}$

$s = q_0$

și δ este dată prin tabelul următor:

q	σ	$\delta(q, \sigma)$
q_0	a	(q_0, R)
q_0	#	$(h, \#)$

Mașina se deplasează la dreapta până când găsește un # și se oprește.

Pentru a specifica starea operațiilor unei mașini Turing, trebuie specificată starea mașinii, conținutul benzii și poziția capului. Conținutul benzii va fi specificat ca un șir, împarțit în trei părți: subșirul (eventual vid) aflat la stânga poziției curent scanate a benzii, simbolul din poziția curentă, și subșirul (eventual vid) aflat la dreapta poziției curente. Pentru a evita situația în care două tupluri să corespundă aceleași combinații a capului și conținutului benzii, impunem condiția ca ultimul șir să nu se termine cu #.

Definiția 1.2

Configurația unei mașini Turing $M = (K, \Sigma, \delta, s)$ este un element din

$(K \cup \{h\}) \times \Sigma^* \times \Sigma \times (\Sigma^* (\Sigma - \#) \cup \{e\})$.

O configurație care conține starea de $halt$ se numește *configurație de oprire*.

Trecerea mașinii Turing dintr-o configurație în alta este specificată prin definiția următoare.

Definiția 1.3

Fie $M = (K, \Sigma, \delta, s)$ o mașină Turing și fie (q_1, w_1, a_1, u_1) și (q_2, w_2, a_2, u_2) configurații ale lui M. Atunci

$(q_1, w_1, a_1, u_1) \vdash_M (q_2, w_2, a_2, u_2)$

dacă și numai dacă, pentru $b \in \Sigma \cup \{L, R\}$, $\delta(q_1, a_1) = (q_2, b)$ fie:

1. $b \in \Sigma$, $w_1 = w_2$, $u_1 = u_2$, și $a_2 = b$;

sau

2. $b = L$, $w_1 = w_2 a_2$ și fie
   (a) $u_2 = a_1 u_1$, dacă $a_1 \neq \#$ sau $u_1 \neq e$ sau
   (b) $u_2 = e$, dacă $a_1 = \#$ și $u_1 = e$;

sau

3. $b = R$, $w_2 = w_1 a_1$, și fie
   (a) $u_1 = a_2 u_2$, sau
   (b) $u_1 = u_2 = e$ și $a_2 = \#$

În cazul 1, M rescrie un simbol în poziția curentă. În cazul 2, M își deplasează capul o poziție la stânga; dacă se mută la stânga benzii neocupate, simbolul # din poziția curentă dispare din configurație. În cazul 3, M își deplasează capul o poziție la dreapta; dacă se mută pe banda neocupată un nou simbol # apare în configurație, ca fiind noul simbol scanat.

Pentru a simplifica notația vom descrie o configurație (q, w, a, u) prin $(q, w\underline{a}u)$, unde simbolul din poziția curentă este subliniat.

Exemplu 1.3

Fie șirurile $w, u \in \Sigma^*$, unde u nu se termină cu #, și fie $a, b \in \Sigma$.

Cazul 1. $\delta(q_1, a) = (q_2, b)$

$(q_1, w\underline{a}u) \vdash_M (q_2, w\underline{b}u)$

Cazul 2. $\delta(q_1, a) = (q_2, L)$

(a) $(q_1, wb\underline{a}u) \vdash_M (q_2, w\underline{b}au)$

(b) $(q_1, wb\underline{\#}) \vdash_M (q_2, w\underline{b})$

Cazul 3. $\delta(q_1, a) = (q_2, R)$

(a) $(q_1, w\underline{a}bu) \vdash_M (q_2, wa\underline{b}u)$

(b) $(q_1, w\underline{a}) \vdash_M (q_2, wa\underline{\#})$

Se observă că dacă $b = L$ și $w = e$, atunci (q_1, w_1, a_1, u_1) nu duce la nicio configurație, pentru că nu există $w_2 \in \Sigma^*$ și $a_2 \in \Sigma$ astfel încât $w_1 = w_2a_2$. O astfel de configurație este numită configurație de agățare.

Definiția 1.4

Pentru orice mașină Turing M, $\vdash_M^*$ este închiderea reflexivă și tranzitivă a relației $\vdash_M$; din configurația C_1 se ajunge în configurația C_2 în zero sau mai mulți pași dacă $C_1 \vdash_M^* C_2$. Un calcul efectuat de M este o secvență de configurații $C_1, C_2, ..., C_n$ astfel încât

$$C_0 \vdash_M C_1 \vdash_M C_2 \vdash_M ... \vdash_M C_n$$

Spunem că lungimea calculului este n sau că are n pași.

1.2. Utilizarea mașinii Turing

Pentru a putea prezenta puterea de calcul a mașinii Turing trebuie să stabilim câteva convenții de utilizare a mașinii. Considerăm că șirul de intrare este încadrat de câte un simbol # și este plasat la limita stânga a benzii de intrare; capul de citire / scriere este poziționat pe simbolul # care delimitează capătul dreapta al șirului; mașina începe operațiile din starea inițială.

Fie $M = (K, \Sigma, \delta, s)$ o mașină Turing și $w \in \Sigma^*$. M se oprește pe intrarea w, dacă și numai dacă pornind din configurația inițială $(s, \#w\underline{\#})$ în urma operațiilor efectuate asupra lui w, mașina ajunge într-o configurație de oprire (configurația care conține starea h).

Pentru început vom analiza calculabilitatea cu mașina Turing în termenii funcțiilor definite pe șiruri.

Definiția 1.5

Fie Σ_0 și Σ_1 alfabete care nu conțin simbolul $\#$. Fie $f : \Sigma_0 \to \Sigma_1$ o funcție. Mașina Turing $M = (K, \Sigma, \delta, s)$ calculează funcția f, dacă pentru $\Sigma_0, \Sigma_1 \subseteq \Sigma$, $\forall\ w \in \Sigma_0^*$, $f(w) = u$, atunci

$$(s, \#w\underline{\#}) \vdash_M^* (h, \#u\underline{\#})$$

Dacă o astfel de mașină Turing există, atunci f este o funcție calculabilă Turing.

Astfel, când M are pe banda de intrare șirul w conform convenției prezentate, M trebuie să se oprească cu șirul $f(w)$ pe bandă, încadrat de $\#$, cu capul la dreapta valorii calculate. Restul benzii trebuie să conțină doar $\#$. Nu se garantează comportamentul mașinii dacă nu se respectă convențiile prezentate.

O mașină Turing care calculează o funcție nu se agață pentru nicio intrare din domeniul funcției. Acest lucru este important pentru că putem presupune că o mașină care calculează o funcție nu încearcă să consulte simboluri aflate la stânga șirului de intrare.

De notat că nu se dau asigurări despre comportarea mașinii Turing dacă nu satisface specificația, de exemplu, dacă şirul de intrare conţine simboluri din $\Sigma - \Sigma_0$.

Exemplu 1.4

Fie $\Sigma_0 = \Sigma_1 = \{a, b\}$ şi fie $f : \Sigma_0^* \to \Sigma_1^*$ definită astfel:

$\forall\ w \in \Sigma_0^*$, $f(w) = \overline{w}$

unde $\overline{w}$este rezultatul înlocuirii fiecărei apariţii a simbolului a în w prin b şi invers.

Funcția f este calculată de o mașină Turing M care parcurge intrarea de la dreapta la stânga, interschimbând simbolurile a cu b, până când simbolul $\#$ care delimitează capătul stânga al șirului este găsit. Pentru a lăsa banda în forma standard, M trebuie să poziţioneze capul pe simbolul $\#$ care se află la limita dreapta a șirului.

Fie $M = (K, \Sigma, \delta, s)$ unde

$K = \{q_0, q_1, q_2\}$,

$\Sigma = \{a, b, \#\}$,

$s = q_0$

şi δ este definită prin următorul tabel:

q	σ	$\delta(q, \sigma)$
q_0	a	(q_1, L)
q_0	b	(q_1, L)
q_0	$\#$	(q_1, L)
q_1	a	(q_0, b)
q_1	b	(q_0, a)
q_1	$\#$	(q_2, R)
q_2	a	(q_2, R)
q_2	b	(q_2, R)
q_2	$\#$	$(h, \#)$

De exemplu:

$$
\begin{aligned}
(q_0, \#aab\#) &\vdash_M (q_1, \#aa\underline{b}) \\
&\vdash_M (q_0, \#aaa) \\
&\vdash_M (q_1, \#a\underline{a}a) \\
&\vdash_M (q_0, \#a\underline{b}a) \\
&\vdash_M (q_1, \#\underline{a}ba) \\
&\vdash_M (q_0, \#\underline{b}ba) \\
&\vdash_M (q_1, \underline{\#}bba) \\
&\vdash_M (q_2, \#\underline{b}ba) \\
&\vdash_M (q_2, \#b\underline{b}a) \\
&\vdash_M (q_2, \#bb\underline{a}) \\
&\vdash_M (q_2, \#bba\underline{\#}) \\
&\vdash_M (h, \#bba\underline{\#})
\end{aligned}
$$

De notat de asemenea că M calculează corect valoarea $f(e) = e$:

$$
\begin{aligned}
(q_0, \#\#) &\vdash_M (q_1, \underline{\#}\#) \\
&\vdash_M (q_2, \#\underline{\#}) \\
&\vdash_M (h, \#\underline{\#})
\end{aligned}
$$

Dacă M porneşte dintr-o configuraţie ca $(q_0, \#\#ab\underline{\#}\#ab)$ rezultatul obținut de mașină este irelevant pentru întrebarea dacă M calculează f.

Cum $(q_0, \#w\#) \vdash_M^* (h, \#\overline{w}\#)$, $\forall\ w \in \Sigma_0^*$, M calculează f şi f este calculabilă Turing.

Noţiunea de funcţie calculabilă Turing pe şiruri poate fi extinsă în mai multe moduri. Putem considera funcţii cu orice număr de argumente, inclusiv zero. Astfel, fie f o funcţie, $f : (\Sigma_0^*)^k \to \Sigma_1^*$, $k \geq 0$, o maşină Turing $M = (K, \Sigma, \delta, s)$, unde $\Sigma_0, \Sigma_1 \subseteq \Sigma$. $\forall\ w_1, \ldots, w_k \in \Sigma^k$ dacă $f(w_1, \ldots, w_k) = u$, atunci $(s, \#w_1\#w_2\#\ldots\#w_k\underline{\#}) \vdash_M^* (h, \#u\underline{\#})$. Spunem că M calculează f, respectiv f este calculabilă Turing.

O altă extindere posibilă este cea a funcţiilor calculabile Turing definite pe mulţimea numerelor naturale.

Fie I un simbol diferit de #. Un număr natural n poate fi reprezentat (în notaţie unară) prin şirul I^n (0 este reprezentat prin şirul vid).

O funcţie $f : N \to N$ este calculată de o mașină Turing M, dacă M calculează funcţia $f' : \{I\}^* \to \{I\}^*$, unde $f'(I^n) = I^{f(n)}$, $n \in N$. Adică o mașină Turing calculează o funcţie de la numere la numere, calculând funcţia corespunzătoare de la notaţii unare la notaţii unare. Mai general, o mașină Turing M calculează o funcţie $f : N^k \to N$, dacă, calculează funcţia $f' : (\{I\}^*)^k \to \{I\}^*$, unde $f'(I^{n1}, \ldots, I^{nk}) = I^{f(n1 \ldots nk)}$, $\forall\, n_1, \ldots, n_k \geq 0$.

O funcţie definită pe numere sau pe k – tupluri de numere este calculabilă Turing, dacă există o mașină Turing care o poate calcula.

Exemplu 1.5

Fie f funcţia succesor: $f(n) = n+1$, $n \in N$.

Proiectăm o mașină Turing M, care calculează f. Mașina scrie un simbol I în poziția curentă a benzii, mută capul o poziție la dreapta şi se oprește.

Formal, $M = (K, \Sigma, \delta, s)$ unde

$K = \{q_0\}$,

$\Sigma = \{I, \#\}$,

$s = q_0$

şi δ este specificată prin tabelul:

q	σ	$\delta(q, \sigma)$
q_0	I	(h, R)
q_0	$\#$	(q_0, I)

De exemplu,

$(q_0, \#I\underline{I}\#) \vdash_M (q_0, \#I\underline{I}I) \vdash_M (h, \#I\underline{I}\#)$

În cazul general,

$(q_0, \#I^n\#) \vdash_M^* (h, \#I^{n+1}\underline{\#})$.

De notat în particular cazul $n = 0$

$(q_0, \#\underline{\#}) \vdash_M^* (q_0, \#I) \vdash_M (h, \#\underline{I}\#)$

Astfel, M calculează funcţia $f' : \{I\}^* \to \{I\}^*$ astfel încât $f'(I^n) = I^{n+1}$, $\forall$ n $\in$ N, deci M calculează funcţia succesor f : N $\to$ N.

O altă consecinţă importantă a noţiunii de funcţie calculabilă Turing este aceea de limbaj decidabil Turing.

Să considerăm un alfabet Σ_0 care nu conţine simbolul #. Limbajul $L \subseteq \Sigma_0^*$ este decidabil Turing, dacă şi numai dacă funcţia $X_L : \Sigma_0^* \to \{\text{Y}, \text{N}\}$ este calculabilă Turing, unde $\forall\ w \in \Sigma_0^*$

$$X_L(\text{w}) = \begin{cases} \text{Y}, & w \in L \\ \text{N}, & w \notin L \end{cases}$$

Dacă X_L este calculat de o maşină Turing M, atunci spunem că M decide L sau că este o procedură de decizie pentru L.

Exemplu 1.6

Fie $\Sigma_0 = \{a\}$ şi fie $L = \{w \in \Sigma_0^* : |w| \text{ este pară}\}$.

Atunci următoarea maşină Turing $M = (K, \Sigma, \delta, s)$ este o procedură de decizie pentru L:

$K = \{q_0, \ldots, q_6\}$

$\Sigma = \{a, \text{Y}, \text{N}, \#\}$

$s = q_0$

şi δ este definită prin tabelul:

q	σ	$\delta(q, \sigma)$
q_0	#	(q_1, L)
q_1	a	$(q_2, \#)$
q_1	#	(q_4, R)
q_2	#	(q_3, L)
q_3	a	$(q_0, \#)$
q_3	#	(q_6, R)
q_4	#	$(q_5, \underline{Y})$
q_5	$\underline{Y}$	(h, R)
q_5	$\underline{N}$	(h, R)
q_6	#	$(q_5, \underline{N})$

Intrările omise sunt nerelevante şi pot fi definite arbitrar.

M şterge şirul de intrare de la dreapta la stânga, păstrând paritatea numărului de simboluri pe care le-a şters. Stările q_0, q_1, q_2, q_3 sunt utilizate în acest scop. La descoperirea simbolului # care delimitează capătul stânga al intrării, M trece din starea q_1 în q_4 sau din q_3 în q_6, scrie $\underline{Y}$ sau $\underline{N}$, mută capul o poziţie la dreapta şi se opreşte. Astfel:

$(q_0, \#a^n\#) \vdash_M^* (h, \# \underline{Y} \#)$, n este par;

$(q_0, \#a^n\#) \vdash_M^* (h, \# \underline{N} \#)$, n este impar;

Prin urmare M calculează X_L, deci L este decidabil Turing.

O altă modalitate de utilizare a maşinii Turing este ca acceptor de limbaj.

Fie Σ_0 un alfabet care nu conţine #. Spunem că o maşină Turing M acceptă şirul $w \in \Sigma_0^*$, dacă M se opreşte pentru intrarea w. Astfel M acceptă limbajul $L \subseteq \Sigma_0^*$, dacă şi numai dacă

$L = \{ w \in \Sigma_0^* : \text{M acceptă } w \}$.

Limbajul este acceptat Turing dacă există o anumită maşină Turing care să-l accepte. Vom vedea că orice limbaj decidabil Turing este acceptat Turing, dar nu şi invers.

Exemplu 1.7

Fie $\Sigma_0 = \{a, b\}$ şi fie $L = \{w \in \Sigma_0^*$: w conţine cel puţin un simbol $a\}$. Atunci L este acceptat de maşina Turing $M = (K, \Sigma, \delta, s)$, unde

$K = \{q_0\}$

$\Sigma = \{a, b, \#\}$

$s = q_0$

şi δ este definită prin tabelul:

q	σ	$\delta(q, \sigma)$
q_0	a	(h, a)
q_0	b	(q_0, L)
q_0	$\#$	(q_0, L)

Pornind din configuraţia $(q_0, \#w\#)$, $w \in \Sigma_0^*$, maşina se deplasează la stânga până când găseşte un simbol a şi se opreşte. Dacă nu găseşte niciun simbol a maşina se agaţă, prin urmare nu se opreşte. Astfel L este exact mulţimea şirurilor $w \in \Sigma_0^*$ cu proprietatea că M se opreşte pentru intrarea w, deci M acceptă L.

1.3. Compunerea maşinilor Turing

Scopul acestei secţii este de a prezenta o metodă pentru compunerea maşinilor Turing, astfel încât maşinile Turing pot fi văzute ca module sau subrutine ale altor maşini Turing.

Lema 1.1

Fie M o maşină Turing şi fie $(q_i, w_i\underline{a}_i u_i)$, $(i = 1, 2, 3)$ configuraţii ale lui M.
Atunci, dacă

$(q_1, w_1\underline{a}_1 u_1) \vdash_M^* (q_2, ww_2\underline{a}_2 u_2)$ pentru un şir w, şi

$(q_2, w_2\underline{a}_2 u_2) \vdash_M^* (q_3, w_3\underline{a}_3 u_3)$

atunci

$(q_1, w_1\underline{a}_1u_1) \vdash_M^* (q_3, ww_3\underline{a}_3u_3)$

Demonstrație. Această lemă, afirmă în esenţă, că dacă o maşină Turing poate realiza anumite operaţii fără a-şi muta capul la stânga unei anumite poziții, atunci va realiza aceste operaţii, indiferent dacă este cea mai din stânga poziție a benzii sau o poziție oarecare de pe bandă. În cazul din urmă, conţinutul părţii stânga a benzii rămâne neschimbat.

Cheia demonstraţiei este că dacă M nu se agaţă în timpul operaţiilor $(q_2, w_2\underline{a}_2u_2) \vdash_M^* (q_3, ww_3\underline{a}_3u_3)$, cum M nu are o modalitate directă de a detecta capătul stânga al benzii, nu trebuie să facă nicio încercare de a-şi muta capul la stânga primului simbol din $w_2a_2u_2$. Apoi, dacă porneşte în configuraţia $(q_2, ww_2\underline{a}_2u_2)$ va ajunge în configurația $(q_3, ww_3\underline{a}_3u_3)$ fără a pătrunde în primele $|w|$ pătrate benzii. ■

Această lemă este utilă pentru compunerea maşinilor Turing văzute ca subrutine. Să presupunem că o maşină Turing M_1 a fost proiectată pentru a se comporta într-un anumit mod, în particular, se ştie că M_1 nu se agaţă atunci când intrarea este conform convenţiilor prezentate. Atunci M_1 poate fi utilizată ca o componentă a unei alte maşini Turing. M_2 pregăteşte un anumit şir ca intrare pentru M_1, plasându-l la limita dreapta a porţiunii nevide a benzii, cedează controlul lui M_1 cu capul de citire / scriere poziționat la dreapta şirului. La sfârşit primeşte controlul de la M_1 când aceasta termină de calculat. Rezultatele operaţiilor parţiale efectuate de M_2 înainte de a ceda controlul lui M_1 vor fi păstrate în timpul operaţiilor lui M_1, cu condiţia ca aceste rezultate parţiale să fi fost salvate pe bandă, la stânga poziției de pe bandă unde intrarea lui M_1 începe. Este garantat că M_1 nu va invada rezultatele parţiale salvate de M_2, pentru că M_1 nu îşi va muta capul la stânga simbolului # care marchează capătul stânga al intrării sale.

Vom presupune că maşinile care sunt compuse nu au nicio stare în comun. Ca şi în cazul altor automate (automate finite, cu stiva) operaţiile unei maşini Turing nu depind de alegerea particulară a elementelor utilizate ca stări, ci doar de modul în care acestea sunt legate unele de altele prin funcţiile de tranziţie.

Putem acum să descriem construcţia unor maşini Turing simple şi modul în care pot fi compuse pentru a forma maşini mai puternice. Considerăm un alfabet Σ.

Maşini de bază. Sunt de două tipuri: *scriere*, *deplasare*.

1. Există $|\Sigma|$ *maşini scriere*, câte una pentru fiecare simbol din Σ. Fiecare maşină *scrie* un anumit simbol în poziţia curentă şi apoi se opreşte. Conţinutul anterior al poziției benzii este ignorat şi capul nu este mutat.

Formal, maşina Turing care scrie simbolul a este $w_a=(K, \Sigma, \delta, s)$ unde $K=\{q\}$, pentru o stare oarecare q, $s = q$ şi $\delta(q, b) = (h, a)$, $h \in \Sigma$. Pentru că aceste maşini sunt utilizate foarte des, le vom nota cu a (în loc de w_a).

2. Maşini *deplasare*, mută capul o poziţie la stânga sau la dreapta şi apoi se opresc. Fie q o anumită stare fixată. Fie $V_L=(\{q\}, \Sigma, \delta_L, q)$ unde $\delta_L(q, a)=(h, L)$, $\forall\ a \in \Sigma$. Fie $V_R=(\{q\}, \Sigma, \delta_R, q)$ unde $\delta_R(q, a) = (h, R)$, $\forall\ a \in \Sigma$. Le vom nota cu L, R.

Reguli de compunere a maşinilor Turing. Maşinile Turing vor fi compuse într-un mod sugestiv pentru structura unui automat finit (AF). Maşinile individuale sunt similare stărilor unui AF, şi vor fi conectate unele cu altele analog conectării stărilor unui AF. Conectarea unei maşini la alta nu este realizată până când prima maşină nu se opreşte; cealaltă maşină porneşte din starea ei iniţială cu banda şi poziţia capului aşa cum au fost lăsate de prima maşină.

Astfel, dacă M_1 şi M_2 sunt maşini Turing, maşina din figură

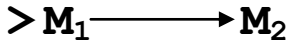


Figura 1-2

funcţionează astfel: porneşte din starea iniţială a lui M_1, funcţionează ca M_1 până când M_1 s-ar fi oprit; iniţiază M_2 şi funcţionează ca M_2.

Conexiunea dintre două maşini poate fi contingentă simbolului din poziţia curentă la oprirea primei maşini (aşa cum conexiunea dintre două stări ale unui automat finit poate depinde de următorul simbol citit). Astfel, dacă M_1, M_2, M_3 sunt maşini Turing, atunci maşina compusă începe prin a iniţia M_1; dacă M_1 se opreşte, este iniţiată M_2 sau M_3 în funcţie de conţinutul poziţiei curente în acel moment (a sau b).

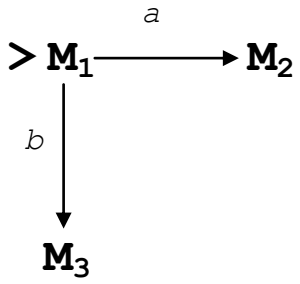


Figura 1-3

Definiţia 1.6

O schemă de maşină este un triplet $(\mathcal{m}, \eta, M_0)$, unde

$\mathcal{m}$ este o mulţime finită de maşini Turing cu un alfabet comun Σ şi mulţimi de stări disjuncte;

$M_0 \in \mathcal{m}$ este maşina iniţială;

$\eta : \mathcal{m} \times \Sigma \to \mathcal{m}$ este o funcţie.

O schemă $(\mathcal{m}, \eta, M_0)$ reprezintă o maşină Turing compusă din maşinile individuale $M \in \mathcal{m}$. Maşină compusă – notată $\overline{M}$, îşi începe operaţiile simulând pe cele ale maşinii iniţiale M_0. Când M_0 se opreşte, $\overline{M}$ începe operaţiile unei alte maşini din $\mathcal{m}$. În particular, dacă M_0 s-ar fi oprit în timp ce scana simbolul a, atunci:

- dacă $\eta(M_0, a)$ este nedefinit, $\overline{M}$ se opreşte
- dacă $\eta(M_0, a) = M \in \mathcal{m}$, atunci $\overline{M}$ continuă operaţiile din starea iniţială a lui M.

Definiţia 1.7

Fie $\mathcal{m} = \{M_0, ..., M_m\}$ $(m \geq 0)$, $i = 0 \ldots m$, $M_i = (K_i, \Sigma, \delta_i, s_i)$. Fie $q_0, ..., q_m$ stări noi care nu sunt în niciuna dintre mulţimile K_i. Atunci dacă $(\mathcal{m}, \eta, M_0)$ este o schemă de maşini Turing, ea reprezintă maşina Turing $M = (K, \Sigma, \delta, s)$, unde

$K = K_0 \cup \ldots \cup K_m \cup \{ q_0, \ldots, q_m \}$

$s = s_0$

și δ este definită astfel:

(a) dacă $0 \le i \le m$, $q \in K_i$, $a \in \Sigma$, $\delta_i(q, a) = (p, b)$, $p \ne h$ atunci $\delta(q, a) = \delta_i(q, a) = (p, b)$

(b) dacă $0 \le i \le m$, $a \in \Sigma$, $q \in K_i$, $a \in \Sigma$, $\delta_i(q, a) = (h, b)$ atunci $\delta(q, a) = (q_i, b)$

(c) dacă $0 \le i \le m$, $a \in \Sigma$, $\eta(M_i, a)$, nu este definit, atunci $\delta(q_i, a) = (h, a)$

(d) dacă $0 \le i \le m$, $a \in \Sigma$, $\eta(M_i, a) = M_j$, $\delta_j(s_j, a) = (p, b)$, atunci

$$\delta(q_i, a) = \begin{cases} (p, b), p \ne h \\ (q_j, b), p = h \end{cases}$$

Stările $q_0, \ldots, q_m$ sunt introduse ca puncte de oprire temporare între diferite mașini; q_i semnifică starea de *halt* a mașinii M_i. În definiția funcției δ, punctul (a) afirmă că fiecare mașină funcționează așa cum este de așteptat până când este gata să se oprească; conform punctului (b) mașina M_i intră în starea q_i în loc să se oprească; punctele (c) și (d) menționează că din starea q_i mașina compusă fie se oprește fie continuă primul pas al mașinii M_j în funcție de conținutul poziției curente. De notat că mulțimile de stări K_i sunt disjuncte ceea ce denotă că δ este o funcție.

Exemplu 1.8

Fie m mașina R definită anterior; $m = \{M_0\}$ unde $M_0 = R = \{(q), \Sigma, \delta_R, q\}$. Fie schema (m, η, M_0) unde

$$\eta(M_0, a) = \begin{cases} M_0, a \ne \# \\ \text{nedefinit}, a = \# \end{cases}$$

Atunci (m, η, M_0) reprezintă mașina $R_\#$, unde

$R_\# = \{(q, q_0), \Sigma, \delta, q\}$

$$\delta(q, a) = (q_0, R), a \in \Sigma$$

şi

$$\delta(q_0, a) = \begin{cases} (q_0, R), a \neq \# \\ (h, a), a = \# \end{cases}$$

Această maşină, pornind din orice poziţie a benzii, îşi mută capul o poziţie la dreapta, şi continuă să îşi mute capul la dreapta, până găseşte un simbol # şi se opreşte.

Dacă $\Sigma = \{\#, a, b, c\}$, atunci $R_\#$ poate fi reprezentată astfel:

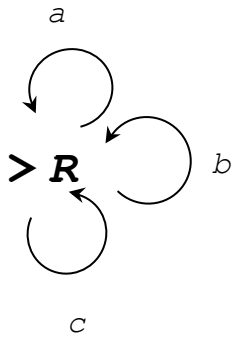


Figura 1-4

Regulile de bază pentru reprezentarea unei scheme (m, η, M_0) sunt următoarele:

1. fiecare maşină Turing $M \in m$ este reprezentată numai o dată
2. maşina iniţială M_0 este marcată cu o săgeată >
3. ori de câte ori $a \in \Sigma$ şi $\eta(M, a)$ este definită, de exemplu, dacă $\eta(M, a) = M'$ o săgeată etichetată cu a este reprezentată între M şi M'

Pentru că aceste reguli pot fi complicat de aplicat, le vom simplifica. Permitem fi două sau mai multe apariţii ale aceleiaşi maşini Turing, care reprezintă copii ale aceleaşi maşini.

Exemplu 1.9

Maşina compusă m constă din două copii ale lui R.

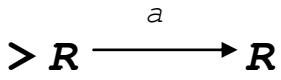


Figura 1-5

Maşina reprezentată în diagramă, îşi mută capul o poziţie la dreapta, apoi, dacă pătratul conţine *a*, îşi mută capul o poziţie la dreapta.

Săgeţile multiple şi subdiagramele similare pot fi eliminate în moduri diferite. O săgeată etichetată cu diferite simboluri înlocuieşte mai multe săgeţi; o săgeată neetichetată corespunde săgeţilor pentru fiecare simbol din Σ.

Exemplu 1.10

Fie Σ = (*a*, *b*, *c*, *#*) , diagrama

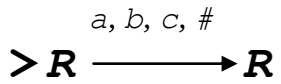


Figura 1-6

poate fi reprezentată

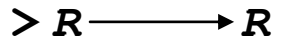


Figura 1-7

Maşina îşi deplasează capul la dreapta două poziţii şi se opreşte. Eliminând arcul, putem scrie *RR* sau R^2. În general R^k reprezintă maşina MM ... M cu *k* apariţii ale lui M.

În continuare vom utiliza notaţia $\overline{\sigma}$ pentru a desemna orice simbol diferit de σ. Cu această notaţie maşina $R_{\#}$ poate fi reprezentată ca:

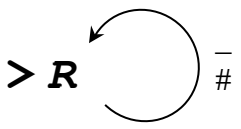
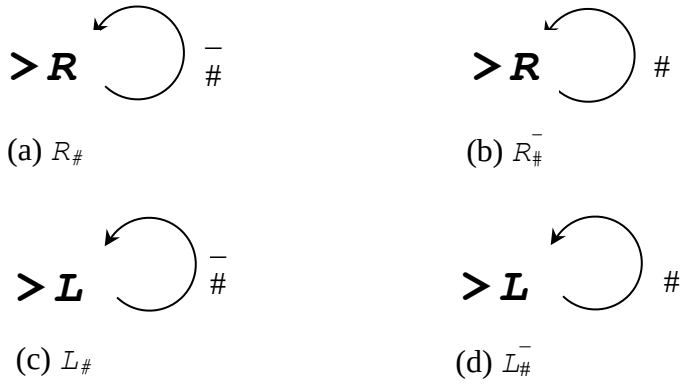


Figura 1-8

deplasează capul la dreapta până la *#*.

Exemplu 1.11

Reprezentarea maşinilor care se deplasează până la un anumit simbol:



(a) $R_{\#}$ (b) $R_{\#}^{-}$

(c) $L_{\#}$ (d) $L_{\#}^{-}$

Figura 1-9

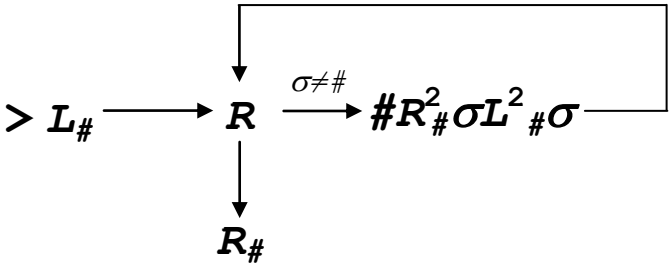
(a) $R_{\#}$ găseşte prima poziție liberă la dreapta celei curente (se deplasează dreapta până la $\#$)
(b) $R_{\#}^{-}$găseşte prima poziție utilizată la dreapta celei curent scanate (se deplasează dreapta până la primul simbol diferit de $\#$)
(c) $L_{\#}$ găseşte prima poziție liberă la stânga celei curente (se deplasează stânga până la $\#$)
(d) $L_{\#}^{-}$ găseşte prima poziție utilizată la stânga celei curent scanate (se deplasează stânga până la primul simbol diferit de $\#$)

Exemplu 1.12

Să reprezentăm maşina de copiere C care primește ca intrare un şir w încadrat de simboluri $\#$, cu capul pe simbolul $\#$ care marchează sfârşitul lui w şi lasă pe bandă şirul $w\#w$ încadrat de simboluri $\#$ cu capul pe simbolul $\#$ care marchează sfârşitul şirului $w\#w$. Prin convenție şirul w, posibil vid, conţine numai simboluri diferite de $\#$.

Formal, dacă s este starea iniţială a lui C, atunci

$(s, \#w\underline{\#}) \vdash_C^* (h, \#w\#w\underline{\#})$.



#

Figura 1-10

Să urmărim operațiile lui C pentru intrarea $\#abc\#$. După $L_{\#}$, pe bandă vom avea șirul $\underline{\#}abc\#$ cu capul poziționat pe simbolul # care delimitează stânga șirul. Capul este mutat o poziţie la dreapta prin execuția mașinii *R* obținându-se $\#\underline{a}bc\#$. Simbolul curent nu este # astfel încât este scris un # în locul lui. Al doilea simbol # la dreapta celui curent scanat este găsit prin execuția mașini R^2 ($\#\#bc\#\underline{\#}$) şi simbolul a este scris în locul lui ($\#\#bc\#\underline{a}$). Apoi capul este poziționat pe al doilea # la stânga simbolului curent prin execuția mașinii L^2 și un a este scris în locul lui ($\#\underline{a}bc\#a$). Acest ciclu se reia până cand se ajunge la sfârșitul șirului inițial.

Câteva dintre configuraţiile mașinii sunt următoarele:

$\#a\underline{b}c\#a$

$\#a\underline{\#}c\#a$

$\#a\#c\#a\underline{\#}$

$\#a\#c\#a\underline{b}$

$\#a\underline{\#}c\#ab$

$\#a\underline{b}c\#ab$ sfârşitul unui ciclu

$\#ab\underline{c}\#ab$

.

.

.

$\#ab\underline{c}\#abc$ sfârşitul celui de al 3 lea ciclu

În acest punct, la mutarea dreapta a capului este găsit un simbol # ($\#abc\underline{\#}abc$). Următorul simbol # la dreapta simbolulu curent este găsit ($\#abc\#abc\underline{\#}$) şi maşina se opreşte.

De notat că Lema 1.1 garantează că maşina C va copia ultimul bloc de simboluri diferite de # pe bandă dacă există mai multe astfel de blocuri. De exemplu, dacă u, v, m sunt şiruri de simboluri diferite de #, C transformă

$\#u\#v\#\underline{w\#}$ în $\#u\#v\#w\#\underline{w\#}$. Desigur, C nu va funcţiona corect dacă porneşte cu simboluri diferite de # la dreapta capului.

1.4. Generalizări ale maşinii Turing

Pentru a înțelege mai bine puterea de calcul a mașinii Turing vom încerca să extindem mașina în diferite moduri. Dar, așa cum vom vedea, noile capabilităţi adăugate nu măresc clasa funcţiilor calculabile sau a limbajelor decidabile, ceea ce întărește ideea că mașina Turing este modelul final de calculabilitate. Extensiile sunt următoarele:

1. permitem benzii să fie infinită în ambele sensuri (nu numai la dreapta);
2. permitem mai multe benzi;
3. permitem mai mute capete pe bandă;
4. banda poate fi bidimensională (în loc de unidimensională);

Pentru fiecare tip de extensie, operaţiile maşinii extinse pot fi simulate printr-o mașina Turing standard.

Bandă infinită în ambele sensuri

Banda nu mai are un capăt stânga și un șir de intrare poate fi scris oriunde pe bandă. Operaţiile se desfăşoară ca în cazul unei mașini Turing obișnuite, cu excepţia faptului că există un spaţiu arbitrar atât la dreapta, cât şi la stânga pentru calcule intermediare.

Formal $M = (K, \Sigma, \delta, s)$; o configuraţie este definită similar (q, w, a, u), $q \in K$, $w, u \in \Sigma^*$, $a \in \Sigma$, unde w nu începe cu # şi u nu se termină cu #; conform notaţiei utilizate pentru mașina Turing, o configurație este descrisă ca (q, wau).

Relaţia dintre configuraţii, $\vdash_M$, este similară celei pentru mașina Turing standard. O diferenţă este că dacă $\delta(q, a) = (p, L)$ atunci $(q, \underline{a}u) \vdash_M (p, \underline{\#}au)$. Adică există bandă disponibilă atât la stânga cât şi la dreapta maşinii, prin urmare aceasta nu se poate agăţa. De asemenea, dacă $\delta(q, a) = (p, R)$ atunci $(q, \underline{\#}au) \vdash_M (p, \underline{a}u)$

Sunt astfel de maşini mai puternice decât mașina Turing clasică? Nu. Orice operaţie efectuată de mașina Turing cu bandă infinită în ambele sensuri, poate fi efectuată de mașina Turing standard.

Lema 1.2

Fie $M_1 = (K_1, \Sigma_1, \delta_1, s_1)$ o maşină Turing cu o bandă infinită în ambele sensuri. Atunci există o maşină Turing standard $M_2 = (K_2, \Sigma_2, \delta_2, s_2)$ astfel încât $\forall\ w \in (\Sigma_1 - \{\#\})^*$, următoarele afirmaţii sunt adevărate :

(a) Dacă M_1 se opreşte pentru w,

$$(s_1, w\underline{\#}) \vdash_M^* (h, u\underline{a}v),\ u, v \in \Sigma_1^*,\ a \in \Sigma_1,$$

atunci M_2 se opreşte pentru w

$$(s_2, \#w\underline{\#}) \vdash_M^* (h, \#u\underline{a}v)$$

(b) Dacă M_1 nu se opreşte, nici M_2 nu se opreşte pentru w.

Pentru a da un răspuns ferm la întrebarea dacă maşinile Turing cu bandă infinită în ambele sensuri, sunt mai puternice, trebuie să definim modul de utilizare al acestor maşini. Vom considera cazul funcţiilor definite pe şiruri. Noţiunile de limbaj decidabil şi acceptat Turing sunt definite analog.

Definiţia 1.8

Spunem că o maşina Turing $M = (K, \Sigma, \delta, s)$ cu bandă infinită în ambele sensuri calculează funcţia $f' : \Sigma_0^* \to \Sigma_1^*$, unde Σ_0, Σ_1 sunt alfabete care nu conţin $\#$, dacă şi numai dacă $\forall\ w \in \Sigma_0^*$, dacă $f(w) = u$, atunci $(s, w\underline{\#}) \vdash_M^* (h, u\underline{\#})$.

Teorema 1.1

Orice funcţie care este calculată sau limbaj care este acceptat sau decis de o maşina Turing cu bandă infinită în ambele sensuri este calculată, decis sau acceptat de o maşină Turing standard.

Demonstraţie. Fie M_1 o maşină Turing cu bandă infinită în ambele sensuri care calculează o anumită funcţie, decide sau acceptă un anumit limbaj. Fie M_2 maşina Turing standard corespunzătoare lui M_1 conform Lemei 1.2. Atunci M_2 calculează aceeaşi funcţie, respectiv decide sau acceptă acelaşi limbaj. ■

Mai multe benzi

Pentru orice număr $k > 0$ fixat, o mașină Turing cu k - benzi constă dintr-o unitate de control şi k benzi (limitate stânga). Fiecare bandă este conectată la dispozitivul de control printr-un cap de citire / scriere.

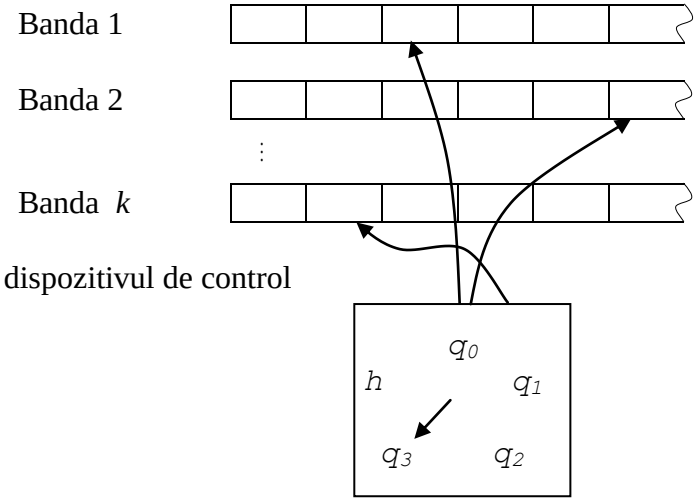


Figura 1-11

Maşina poate într-un singur pas să analizeze toate simbolurile scanate de toate capetele sale şi apoi în funcţie de aceste simboluri şi de starea curentă, poate să rescrie un simbol sau să-şi mute capetele la stânga sau la dreapta.

O mașină Turing cu k - benzi poate fi utilizată pentru a calcula o funcţie sau a decide un limbaj. Convenim ca şirul de intrare să fie plasat pe prima bandă cu capul la limita dreapta. Celelalte benzi sunt iniţial vide, cu capetele la limita stânga a fiecăreia. La sfârşitul operaţiilor, mașina Turing cu k - benzi va avea rezultatul pe prima bandă; conţinutul celorlalte benzi este ignorat.

Cu ajutorul mai multor benzi este deseori mai uşor de construit o mașină Turing pentru o funcţie particulară. Să considerăm copierea unui şir:

$$\#w\underline{\#} \rightarrow \#w\#w\underline{\#},$$

unde w nu conţine simboluri $\#$.

O mașină Turing cu două benzi poate realiza copierea astfel:

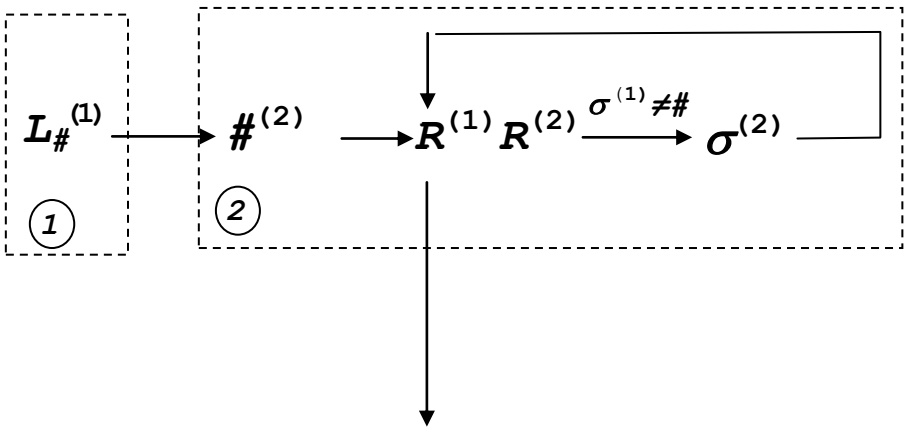
1. mută capul primei benzi la stânga până găseşte un $\#$.

2. mută capetele ambelor benzi la dreapta, copiind fiecare simbol de pe prima bandă pe a doua, până găseşte un # pe prima bandă; prima poziţie a celei de a doua benzi trebuie să conţină #.
3. mută capul celei de a doua benzi la stânga până găseşte un #.
4. mută capete ambelor benzi la dreapta, copiind simboluri de pe a doua bandă pe prima. Se opreşte la găsirea unui # pe a doua bandă.

Secvenţa de acţiuni poate fi reprezentată astfel:

La început:	1*ma* bandă *#w#* 2*a* bandă *#*
După (1)	1*ma* bandă *#w#* 2*a* bandă *#*
După (2)	1*ma* bandă *#w#* 2*a* bandă *#w#*
După (3)	1*ma* bandă *#w#* 2*a* bandă *#w#*
După (4)	1*ma* bandă *#w#w#* 2 bandă *#w#*

Vom ataşa ca indice superior simbolului care reprezintă o maşină numărul benzii pe care operează. De exemplu $\#^{(2)}$ scrie un # pe a doua bandă; $L_{\#}^{(1)}$ caută la stânga un # pe prima bandă. Cu aceste convenţii maşina C de copiere este:



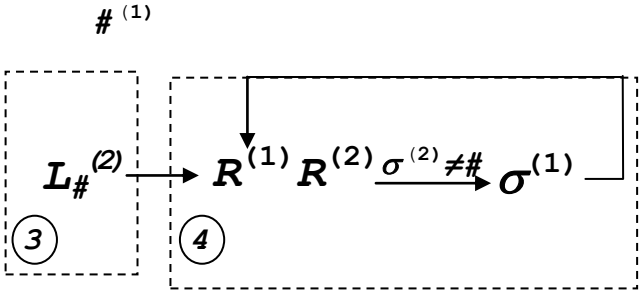


Figura 1-12

Configuraţia unei maşină Turing cu k - benzi este $(q, w_1\underline{a_1}u_1, \ldots, w_k a_k \underline{u_k})$ cu semnificaţia că şirul de pe a i-a bandă este $w_i \underline{a_i} u_i$ cu capul acelei benzi indicând a_i.

Lema 1.3

Fie $k > 0$ şi M_1 o maşină cu k – benzi, alfabetul Σ_1 şi starea iniţială s_1. Atunci există o maşină Turing standard $M_2 = (K_2, \Sigma_2, \delta_2, s_2)$ astfel încât $\Sigma_1 \subseteq \Sigma_2$ şi $\forall\ w \in (\Sigma_1 - \{\#\})^*$,
(a) Dacă M_1 se opreşte pe intrarea w, adică

$$(s_1, \#w\underline{\#}, \underline{\#}, \ldots, \underline{\#}) \vdash_M^* (h, w_1\underline{a_1}u_1, \ldots, w_k\underline{a_k}u_k)$$

pentru $w_1, \ldots, w_K\ u_1, \ldots, u_K$ şi $a_1, \ldots, a_K$, atunci M_2 se opreşte pentru w cu banda având acelaşi conţinut cu prima bandă a lui M_2, respectiv

$$(s_2, \#w\underline{\#}) \vdash_M^* (h, w_1\underline{a_1}n_1)$$

(b) Dacă M_1 se agaţă pentru w, atunci M_2 se agaţă pentru w;
(c) Dacă M_1 nici nu se opreşte nici nu se agaţă pentru w, atunci nici M_2 nu se agaţă nici nu se opreşte pentru w.

Teorema 1.2

Orice funcţie care este calculată sau limbaj care este decis sau acceptat de o maşina Turing cu k – benzi este calculată, decis, respectiv acceptat de o maşină Turing standard.

Demonstraţie. Teorema derivă din Lema 1.3.

Mai multe capete

Într-un pas, pot citi, scrie sau se pot deplasa independent pe aceeași bandă. Trebuie adoptată o anumită convenţie despre ceea ce se întâmplă atunci când două capete încearcă să scrie lucruri diferite pe aceeași poziție a benzii.

Ideea este de a împărți banda în alte benzi, dintre care unele să fie utilizate doar pentru a memora poziția capetelor. Pentru a simula un pas al mașinii cu mai multe capete, banda trebuie scanată de cel puțin două ori. O dată pentru a determina simbolurile indicate de capete, apoi pentru a schimba aceste simboluri sau a deplasa capetele.

Bandă cu două dimensiuni

O altă generalizare a mașinii Turing este de a permite ca banda să aibă două dimensiuni. Mașina are un singur cap care poate fi utilizat pentru citi sau scrie simboluri. Într-un pas, își poate deplasa capul o poziție sus / jos / stânga / dreapta. Intrarea este plasată pe linia de jos a benzii, restul este benzii fiind vidă inițial.

Teorema 1.3

Mașinile Turing cu benzi cu două dimensiuni pot fi simulate de mașini Turing standard.

1.5. Mașina Turing nedeterministă

Dacă automatelor finite li se permite să acţioneze nedeterminist nu rezultă nicio îmbunătăţire a puterii de calcul, dar automatele cu stivă nedeterministe sunt mai puternice decât cele deterministe.

Ne putem imagina şi maşini Turing nedeterministe. Astfel de maşini pot avea pentru anumite combinaţii de stări şi anumite simboluri de intrare mai multe comportamente.

Definiția 1.9

O mașină Turing nedeterministă este un tuplu $M = (K, \Sigma, \Delta, s)$, unde K, Σ, s au aceeași semnificație ca în cazul mașinii Turing standard, iar Δ este o submulţime din

$$(K \times \Sigma) \times ((K \cup \{h\}) \times (\Sigma \cup \{L, R\})).$$

Configuraţiile precum şi relaţiile $\vdash_M$, $\vdash_M^*$ sunt definite în mod analog. Dar în acest caz o configuraţie poate genera mai multe într-un singur pas.

Dacă o mașină Turing acţionează nedeterminist, există o creştere a puterii de calcul? Cum o mașină Turing nedeterminista poate produce două rezultate diferite pentru aceeaşi intrare, trebuie avut grijă la ceea ce este cosiderat a fi rezultatul operaţiilor unei astfel de maşini. Această problemă se rezolvă construind mașinile Turing nedeterministe doar ca acceptori astfel încât singurul fapt semnificativ este dacă maşina se opreşte, nu conţinutul benzii.

Exemplu 1.13

Fie $L = \{w \in \{a, b\}^* : w$ conţine o apariţie a subşirului $abaab\}$. Desigur, L este un limbaj regulat, prin urmare putem construi un automat finit determinist sau nedeterminist pentru a-l accepta.

Maşina Turing primeşte intrarea sub forma $\#w\underline{\#}$, $w \in \{a, b\}^*$. Scanează intrarea de la dreapta la stânga şi alege nedeterminist un punct din w în care să înceapă să verifice subşirul specificat. Din acest moment comportamentul său este determinist. Verifică că următoarele cinci simboluri sunt de la dreapta la stânga, b, a, a, b, a. Dacă acest subşir este detectat, maşina se opreşte şi acceptă şirul de intrare. Dacă nu identifică subşirul, maşina intră într-un ciclu infinit.

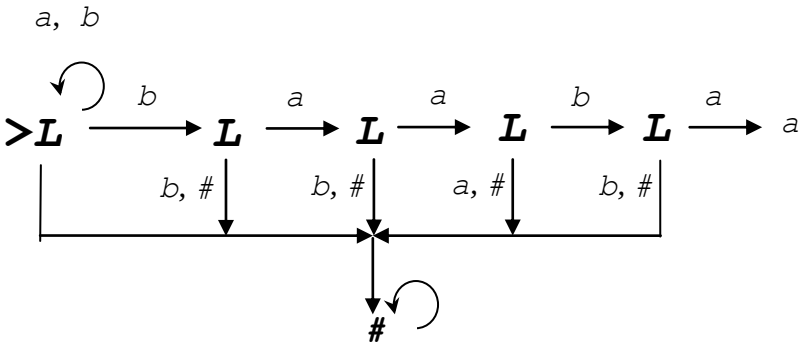


Figura 1-13

Punctul nedeterminist din diagramă este colţul stâng sus, unde dacă simbolul scanat este *b*, există două comportamente posibile. Mută capul spre stânga şi continuă scanare dreapta-stânga sau începe să verifice şirul specificat.

Exemplu 1.14

Un număr compus este un număr care este produsul a două numere naturale fiecare mai mare decât 1; de exemplu, 4, 6, 8, 9, 10, 13 sunt compuse, dar 1, 3, 5, 7, 11, nu sunt.

Fie $L = \{I^n : n$ este un număr compus$\}$. Pentru a proiecta o maşină Turing care să accepte **L**, pare a fi necesar să găsim factorii unui număr. Dacă nedeterminismul este admis, putem proiecta o maşină care să accepte *L*, simplu, ghicind aceşti factori, dacă există. Această maşină operează pentru intrarea $\#I^n\#$ astfel:

1. Alege nedeterminist două numere p, $q > 1$ şi transformă intrarea în $\#I^n\#I^p\#I^q\underline{\#}$
2. Utilizează maşina multiplicativă pentru a transforma rezultatul pasului (1) în $\#I^n\#I^{p\times q}\underline{\#}$.
3. Verifică dacă $I^n = I^{p\times q}$; este uşor de verificat pentru că numai lungimea şirurilor trebuie comparată. Se opreşte dacă lungimile sunt egale, altfel continuă să cicleze la infinit.

Fie *P* - maşina de multiplicare, *E* - maşina care verifica egalitatea a două şiruri: $\forall\ w, u \in \{I\}^*$, E se opreşte pentru intrarea $\#w\#u\underline{\#} \Leftrightarrow$ w=u, şi *G* - maşina care generează un şir de simboluri *I*: scrie I^p, $p \geq 2$. Limbajul *L* este acceptat de maşina *GGPE*.

Maşina *G*:

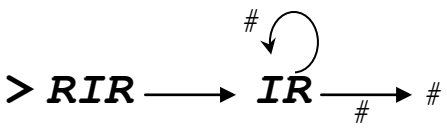
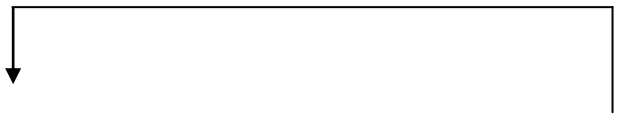


Figura 1-14

Maşina *E*:



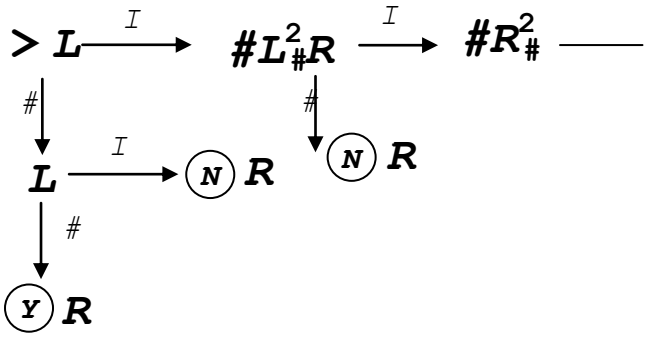


Figura 1-15

Maşina *P*:

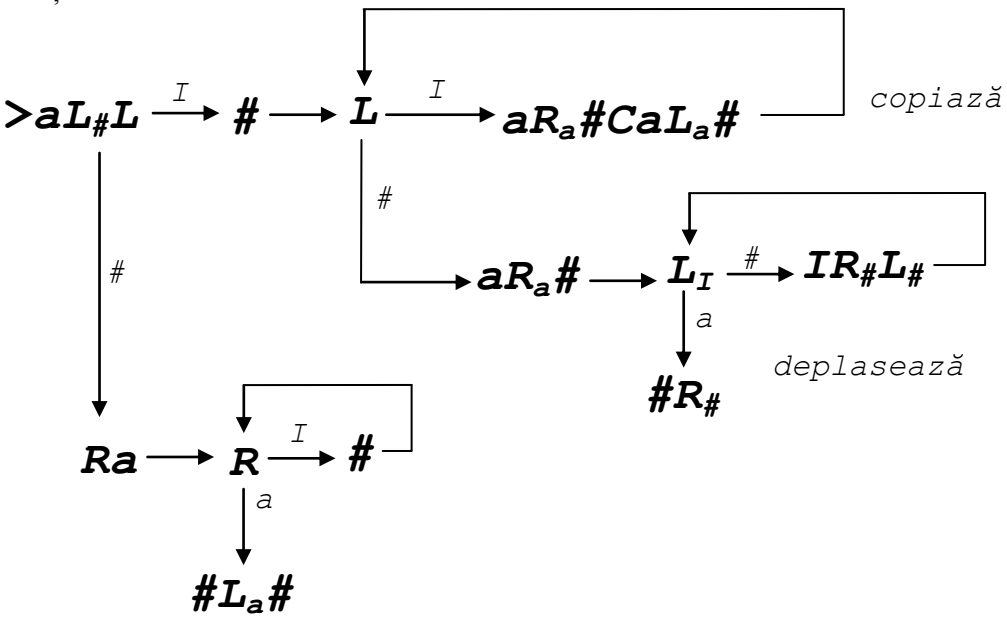


Figura 1-16

Maşina verifică dacă fiecare argument este 0; dacă da, se opreşte cu şirul vid ca rezultat. Altfel, copiază al doilea şir de un număr de ori dat de primul şir; la sfârşit deplasează copiile şi elimină simbolurile #.

Lema 1.4

Pentru orice maşină Turing nedeterministă M_1, putem construi o maşină Turing standard M_2 astfel încât pentru orice şir w care nu conţine #:

(a) Dacă M_1 se opreşte pentru intrarea w, atunci M_2 se opreşte pentru w;

(b) Dacă M_1 nu se opreşte pentru w, atunci nici M_2 nu se opreşte pentru w.

Teorema 1.4

Orice limbaj acceptat de o maşină Turing nedeterministă este acceptat de o maşină Turing deterministă.

1.6. Maşina Turing universală

Una dintre caracteristicile calculatoarelor, pe care maşinile Turing aşa cum au fost prezentate până acum nu o au, este programabilitatea. Până acum, am proiectat o maşină Turing pentru fiecare problemă, în timp ce calculatoarele sunt generale. Vom arăta că şi maşina Turing este în esenţă programabilă.

Pentru început, vom prezenta un mod general de a specifica maşina Turing astfel încât descrierea unei maşini Turing să poată fi folosită ca intrare pentru alte maşini Turing.

O maşină Turing este un tuplu $M = (K, \Sigma, \delta, s)$. Cum K, Σ sunt mulţimi finite, putem scrie acest tuplu ca un şir utilizând simboluri din K şi Σ. Vom codifica stările şi simbolurile de pe bandă ca şiruri peste un alfabet fixat.

Presupunem că există mulţimile infinit numărabile:

$$K_\infty = \{ q_1, q_2, q_3, \ldots \}$$

$$\Sigma_\infty = \{ a_1, a_2, a_3, \ldots \}$$

astfel încât pentru fiecare maşină Turing, mulţimea stărilor este o submulţime finită din K_∞ şi alfabetul de intrare este o submulţime finită din Σ_∞.

Să considerăm următoarea corespondenţă λ între simbolurile componente ale maşinii Turing şi şiruri peste alfabetul $\{I\}$.

σ	$\lambda(\sigma)$
q_i	I^{i+1}
h	I

L	I
R	II
a_i	I^{i+2}

Trebuie remarcat că nu există doi membri din $K_\infty \cup \{h\}$ respectiv din $\{L, R\} \cup \Sigma_\infty$ care să fie reprezentaţi la fel.

Fie c un alt simbol. Vom codifica maşina Turing peste alfabetul $\{c, I\}$.

Fie $M = (K, \Sigma, \delta, s)$, $K \leq K_\infty$, $\Sigma \leq \Sigma_\infty$

$K = \{q_{i_1}, q_{i_2}, \ldots, q_{i_K}\}$, $i_1 < i_2 < \ldots < i_K$

$\Sigma = \{a_{j_1}, a_{j_2}, \ldots, a_{j_1}\}$, $j_1 < j_2 < \ldots < j_l$;

$s = q_{im}$, $1 \leq m \leq k$

Definim kl şiruri S_{pr}, $1 \leq p \leq k$, $1 \leq r \leq l$. Fiecare şir S_{pr} codifică valoarea funcţiei de tranziţie pentru perechea (stare, simbol), respectiv (q_{i_p}, a_{j_r})

$\delta(q_{i_p}, a_{j_r}) = (q', b)$, $q' \in K \cup \{h\}$

$b \in \Sigma \cup \{L, R\}$

$S_{pr} = cw_1cw_2cw_3cw_4c$,

unde $w_1 = \lambda(q_{i_p})$

$w_2 = \lambda(a_{j_r})$

$w_3 = \lambda(q')$

$w_4 = \lambda(b)$

Exemplu 1.15

Fie $M = (K, \Sigma, \delta, s)$, unde

$K = \{q_2\}$

$\Sigma = \{a_1, a_3, a_6\}$

$s = q_2$

$\delta(q_2, a_3) = \delta(q_2, a_6) = (q_2, R)$

$$\delta = (q_2, a_1) = (h, a_3)$$

atunci

$$k = 1,\ i_1 = 2,\ l = 3,\ j_1 = 1,\ j_2 = 3,\ j_3 = 6,$$

şi

$$S_{11} = cI^3cI^3cIcI^5c$$
$$S_{12} = cI^3cI^5cI^3cI^2c$$
$$S_{13} = cI^3cI^8cI^3cI^2c$$

În final, scriem $\rho(M)$ pentru şirul unic

$$cS_0cS_{11}S_{12} \ldots S_{11}S_{21}S_{22} \ldots S_{21} \ldots S_{k1}S_{k2} \ldots S_{kl}c$$

unde $S_0 = \lambda(S)$ este codificarea stării iniţiale.

Acest şir reprezintă codificarea maşinii Turing M. De notat că nu există două şiruri care să corespundă aceleiaşi maşini Turing. Din $\rho(M)$ se poate reconstrui M. $\rho(M)$ are forma cw_0c, …, $w_0 \in \{I\}$; această parte a şirului determină starea iniţială. Restul şirului poate fi spart în blocuri de patru membri din $\{I\}^*$, fiecare bloc fiind separat de vecinii săi prin cc, şi despărţit în interior prin c; din aceste şiruri, valorile δ pot fi reconstruite cu uşurinţă.

Exemplu 1.16

Pentru maşina M utilizată în exemplul anterior, $\rho(M)$ este

$$cI^3ccI^3cI^3cIcI^5ccI^3cI^5cI^3cI^2ccI^3cI^8cI^3cI^2cc$$

Acum putem prezenta maşina Turing universală U, care utilizează codificarea $\rho(M)$ a celorlalte maşini ca programe.

Intuitiv, U primeşte două argumente: o descriere a unei maşini Turing M şi un şir de intrare w, şi execută operaţiile pe care le-ar fi executat M. Desigur, atât M cât şi w, trebuie să fie codificate, la fel va fi şi rezultatul. Şirurile peste alfabetul lui M vor fi codificate astfel:

Dacă

$$w = b_1 \ldots b_n,\ \forall\ b_i \in \Sigma_\infty,\ i = 1 \ldots n$$

atunci

$$\rho(M) = c\lambda(b_1)c\lambda(b_2)c \ldots c\lambda(b_n)c$$

Trebuie remarcat că $\rho(M)$ nu conţine simboluri $\#$. Proprietatea pe care $U = (K_U, \Sigma_U, \delta_U, s_U)$ trebuie să o aibă, poate fi exprimată astfel:

Pentru orice maşină Turing $M = (K, \Sigma, \delta, s)$ şi orice şir $w \in \Sigma^*$

1. dacă $(h, u\underline{a}v)$ este o configuraţie de oprire a lui M astfel încât $(s, \#w\underline{\#}) \vdash_M^* (h, u\underline{a}v)$ atunci $(s_U, \#\rho(M)\rho(w)\underline{\#}) \vdash_U^* (h, \#\rho(uav)\underline{\#})$
2. dacă $(s_U, \#\rho(M)\rho(w)\underline{\#}) \vdash_U^* (h, u'\underline{a}'v')$ pentru o anumită configuraţie de oprire $(h, u'\underline{a}'v')$ a lui U, atunci $a'=\#$, $v'=e$, $u'=\#\rho(uav)$ pentru u, a, v astfel încât $(h, u\underline{a}v)$ este o configuraţie de oprire a lui M şi $(s, \#w\underline{\#}) \vdash_M^* (h, u\underline{a}v)$.

Altfel spus,

1. Dacă M se opreşte pentru w, atunci U se opreşte pentru intrarea formată din codificarea lui M urmată de codificarea lui w şi atunci când U se opreşte, pe bandă există codificarea şirului aflat pe banda lui M, când M se opreşte.
2. Dacă U se opreşte pe intrarea alcătuită din codificarea lui M urmată de codificarea lui w, atunci M se opreşte pe intrarea w, iar conţinutul benzii lui U (la oprirea lui U) conţine codificarea conţinutului benzii lui M când M se opreşte.

Pentru uşurința înțelegerii vom descrie funcționarea unei maşini cu trei benzi U', care este echivalentă cu maşina U (Lema 1.3). U' îşi utilizează benzile astfel:

- 1-ma bandă conţine codificarea benzii lui M
- a 2-a bandă conţine codificarea lui M
- a 3-a bandă conţine codificarea stării lui M la punctul curent al simulării.

Maşina U' porneşte cu şirul $\#\rho(M)\rho(w)$ pe prima bandă şi celelalte benzi goale (este irelevant comportamentul lui U' dacă şirul de intrare nu are această formă). Mai întâi U' pune $\rho(M)$ pe a doua bandă, deplasează $\rho(w)$ la capătul stânga al primei benzi precedându-l de $\#c\rho(\#)$ şi încheindu-l cu $\rho(\#)c$. Deci în acest punct prima bandă conţine $\#\rho(\#w\#)$. (U' nu are nicio dificultate în a determina unde se termină $\rho(M)$ şi începe $\rho(w)$, este singurul punct din $\rho(M)\rho(w)$ unde există trei simboluri c consecutive). Din $\rho(M)$, U' extrage codificarea stării iniţiale a lui M, pe care o copiază pe a treia bandă. Acum U' simulează paşii operaţiilor lui M. Între paşii simulaţi, U' va păstra capetele corespunzătoare benzilor a doua şi a

treia la limita stânga, iar capul primei benzi în dreptul simbolului c care marchează sfârşitul codificării simbolului curent scanat de M. Astfel, dacă M începe scanarea ultimului simbol # din $\#w\#$, U' va începe simularea mutându-şi capul pe prima bandă pe ultimul c de pe această bandă.

Acum U' găseşte pe a doua bandă blocul de forma

$$ccI^icI^jcI^kcI^lcc$$

astfel încât I^i este şirul de simboluri I care se termină la poziţia curentă a capului pe a treia bandă şi I^j este cel de pe prima bandă, şi schimbă prima bandă corespunzător.

Dacă I^l este $\lambda(L)$ sau $\lambda(R)$, aceasta implică mutarea capului cu câteva simboluri la stânga / dreapta; dacă I^l este $\lambda(a)$, $a \in \Sigma_\infty$, atunci poate fi necesară deplasarea părţii dreapta a şirului pe prima bandă într-un sens sau altul pentru a înlocui codificarea unui simbol cu un altul.

În final, U' pune I^k pe a treia bandă şi verifică dacă a treia bandă conţine acum $\lambda(h)$. Dacă nu, atunci U' simulează un alt pas al lui M. Dacă da, U' se opreşte, după ce a mutat capul pe prima bandă la dreapta până la primul #.

2. CALCULABILITATE

Se poate argumenta că dacă reprezentarea limbajelor se face prin şiruri, nu putem reprezenta orice limbaj - există numai o mulţime numărabilă de şiruri peste un alfabet, dar nenumărabilă de limbaje. Automatele finite, automatele cu stivă, maşina Turing, gramaticile sunt exemple de reprezentări finite care pot fi utilizate pentru a specifica limbaje. Automatele şi gramaticile pot fi specificate prin şiruri. Am făcut asta şi pentru maşina Turing. Deci există doar o mulţime numărabilă de limbaje regulate, independente de context, decidabile / acceptate Turing. Deşi am extins maşina Turing, totuşi maşina Turing nu poate accepta / decide decât o parte infinitesimală din toate limbajele posibile. Acum suntem în măsură să spunem ce limbaje nu sunt decidabile / acceptate Turing.

Interesul nostru în găsirea unor astfel de limbaje este justificat de faptul că maşina Turing este formalizarea matematică a noţiunii de algoritm. Deci un limbaj care nu este decidabil Turing, nu este decidabil prin niciun algoritm; nu există niciun mod general şi sistematic de a spune ce şiruri aparţin limbajului.

2.1. Problema opririi maşinii Turing

Teorema 2.1

Orice limbaj decidabil Turing este acceptat Turing.

Demonstraţie. Fie `L` un limbaj decis de o maşină Turing `M`, atunci `L` este acceptat de `M'`:

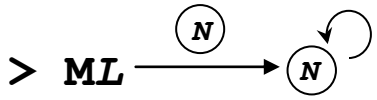


Figura 2-1

care simulează M.
Când M se opreşte, M′ buclează la infinit dacă M (N) lasă pe bandă; dacă M pune (Y) pe bandă, atunci M′ se opreşte. ■

Teorema 2.2

Dacă L este Turing decidabil, atunci $\overline{L}$ (complementul lui L) este Turing decidabil.

Demonstraţie. Dacă L este decis Turing de maşina M, atunci $\overline{L}$ este decis de maşina M'.

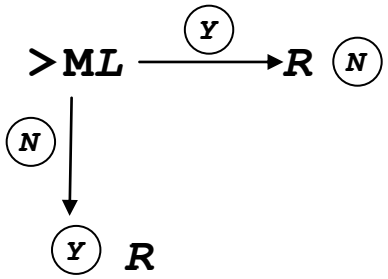


Figura 2-2

M′ simulează M până când se opreşte, apoi interschimbă (Y) cu (N) ■

Dar pentru întrebările:

- este orice limbaj acceptat Turing, decidabil Turing?
- este complementul oricărui limbaj acceptat Turing, acceptat Turing?

Răspunsul în ambele cazuri este NU. Dacă ar fi posibil să se spună pentru orice mașină Turing şi orice şir de intrare dacă respectiva mașină Turing se oprește sau nu pe acea intrare, atunci orice limbaj acceptat Turing ar fi şi decidabil Turing.

Astfel, dacă M_1 ar fi o mașină Turing care acceptă un limbaj L, atunci putem proiecta o mașină Turing M_2 care decide L astfel: M_2 execută operaţiile necesare pentru a prezice eventualul rezultat al operaţiilor lui M_1 pe intrarea w. M_2 s-ar opri cu (Y), (N) pe bandă, în funcţie de acceptarea sau nu a lui w de către M_1. Astfel întrebarea de orice limbaj acceptat Turing este decidabil Turing revine la întrebarea dacă există o super mașină Turing care

poate prezice rezultatul operaţiilor efectuate de maşini Turing arbitrare pe intrări arbitrare. Mai clar, dacă limbajul

$K_0 = \{ \rho(M)\, \rho(w) : M \text{ acceptă şirul } w \}$

este decis de o maşină Turing M_0 atunci orice limbaj acceptat Turing este decidabil Turing.

Atunci pentru o maşină Turing M_1 care acceptă limbajul L, putem construi o maşină Turing, M_2 care decide L, astfel:

1. construim maşina Turing M_1^* care pentru orice şir w, transformă $\#w\underline{\#}$ în $\#\rho(M_1)\,\rho(w)\,\underline{\#}$; adică M_1^* deplasează intrarea peste $|\rho(M_1)|$ poziţii, o codifică peste alfabetul $\{I, C\}$ şi scrie şirul $\#\rho(M_1)$ pe poziţiile cele mai din stânga de benzi. Fie $M_2 = M_1^* M_0$. M_2 transformă intrarea $\#w\underline{\#}$ în $\#\rho(M_1)\,\rho(w)\,\underline{\#}$ şi cedează controlul lui M_0, care, prin ipoteză, decide dacă M_1 acceptă sau nu w.

Mai mult, se poate demonstra că limbajul K_0 este acceptat Turing - este acceptat de o variantă a maşinii Turing universale. Astfel, orice limbaj acceptat Turing este decidabil Turing, dacă şi numai dacă limbajul K_0 acceptat Turing este decidabil Turing.

Dar vom arăta că limbajul K_0 nu este decidabil Turing. Demonstraţia se face prin reducere la absurd. Dacă K_0 ar fi decidabil Turing, atunci

$K_1 = \{ \rho(M) : M \text{ acceptă şirul de intrare } \rho(M) \}$

ar fi decidabil Turing.

Dacă M_0 ar decide K_0, atunci maşina Turing M_1 care să decidă K_1 ar putea fi construită; ar transforma şirul de intrare $\#w\#$ în $\#w\rho(w)\underline{\#}$ şi apoi ar ceda controlul lui M_0.

Acum M_1 va obţine acelaşi rezultat pentru intrarea $\#w\underline{\#}$ ca şi M_0 pentru intrarea $\#w\rho(w)\,\underline{\#}$; dar din definiţia lui K_0, M_0 obţine (Y) pentru intrarea $\#w\rho(w)\,\underline{\#}$, dacă şi numai dacă:

a) w este $\rho(M)$ pentru o maşină Turing M, şi

b) maşina Turing M acceptă şirul w, adică şirul de intrare $\rho(M)$

Dar, aceasta este definiţia lui K_1. Deci este suficient să se demonstreze că limbajul K_1 nu este decidabil Turing.

Dacă limbajul K_1 ar fi decidabil Turing, atunci şi complementul său ar fi decidabil Turing:

$\overline{K_1} = \{ w \in \{I, C\} : w \text{ nu este codificarea } \rho(M) \text{ a niciunei maşini } M, \text{ sau } w = \rho(M) \text{ pentru o anumită maşină } M \text{ care nu acceptă intrarea } \rho(M) \}$

Dar $\overline{K_1}$ nu poate fi nici măcar acceptat Turing. Să presupunem că M^* este o mașină Turing care acceptă $\overline{K_1}$. Este $\rho(M^*)$ în $\overline{K_1}$?

Din definiţia lui $\overline{K_1}$, $\rho(M^*) \in \overline{K_1}$, dacă şi numai dacă M^* nu acceptă şirul de intrare $\rho(M^*)$. Dar se presupune că M^* acceptă $\overline{K_1}$, deci $\rho(M^*) \in \overline{K_1}$ dacă şi numai dacă M^* acceptă $\rho(M^*)$. Ca urmare M^* acceptă $\rho(M^*)$ dacă şi numai dacă M^* nu acceptă $\rho(M^*)$. Ceea ce este absurd, deci presupunerea că există M^* este falsă. ■

Pe scurt, am vrut să vedem dacă orice limbaj acceptat Turing este decidabil Turing. Am văzut că acest lucru ar putea fi adevărat, dacă şi numai dacă limbajul particular K_0 acceptat Turing ar fi decidabil Turing.

Din K_0 am derivat în doi paşi limbajul $\overline{K_1}$, care trebuie să fie decidabil Turing pentru ca limbajul K_0 să fie decidabil Turing. Dar presupunerea că limbajul $\overline{K_1}$ este decidabil Turing duce la o contradicţie.

Am demonstrat teoremele.

Teorema 2.3

Nu orice limbaj acceptat Turing este decidabil Turing.

Teoremă 2.4

Complementele unor limbaje acceptate Turing nu sunt acceptate Turing.

Limbajele K_1 şi K_0 sunt acceptate Turing, dar am arătat că limbajul $\overline{K_1}$ nu este acceptat Turing.

Să revenim asupra aspectului calitativ. Putem vedea K_0 ca un dicţionar infinit care poate răspunde la întrebări de forma: acceptă maşina Turing M şirul w? Cum nu există nicio maşina Turing care să decidă K_0, putem concluziona că nu există niciun algoritm care să decidă pentru o mașină Turing M arbitrară şi un şir w, dacă M acceptă w.

Deci există probleme care nu admit soluţii algoritmice. Astfel de probleme sunt etichetate ca insolvabile. O problemă care poate fi rezolvată de un algoritm este solvabilă. Un algoritm care rezolvă o problemă este numit procedură de decizie pentru acea problemă.

Cea mai faimoasă problemă insolvabilă este cea descrisă de K_0, numită "problema opririi pentru maşina Turing"; ea constă în a determina pentru o maşină Turing arbitrară M şi o intrare w, dacă M se opreşte pentru w.

Acest rezultat de insolvabilitate nu implică însă inexistenţa unor circumstanţe în care este posibilă prezicerea opririi maşinii Turing pentru un şir de intrare.

Putem, de exemplu, examina codificarea maşinii Turing pentru a vedea dacă starea h este reprezentată; dacă nu, maşina nu poate accepta niciun şir de intrare. Aceasta şi analize mai complexe pot oferi anumite informaţii; dar rezultatul secţiei afirmă că astfel de analize sunt fie neconcludente fie dau rezultate proaste. Nu există nicio metodă generală care să decidă corect în toate cazurile.

Mai mult, există anumite maşini particulare pentru care nu există nicio metodă generală de a spune, pentru un şir de intrare, dacă maşina va accepta sau se va opri pentru acel şir de intrare. Aceasta rezultă din faptul că limbajul K_0 este acceptat Turing dar nu este decidabil Turing.

Dacă M_0 este o maşină Turing care acceptă K_0, atunci nu există nicio maşină Turing şi deci niciun algoritm care să determine dacă un şir de intrare oarecare w este acceptat de M_0 sau nu.

Pentru a rezuma, această secţiune stabileşte insolvabilitatea pentru două probleme particulare.

Teorema 2.5

a) Nu există niciun algoritm care să determine, dată fiind o maşină Turing M şi un şir w, dacă M acceptă w.
b) Pentru o anumită maşină Turing fixată M_0, nu există niciun algoritm care să determine pentru un şir w, dacă M_0 acceptă w.

Să subliniem distincţia dintre (a) şi (b). Cum K_0 este $\{\rho(M)\rho(w) : M \text{ acceptă } w\}$, K_0 poate fi spart în infinit de multe submulţimi $K_M = \{\rho(M)\rho(w) : M \text{ acceptă } w\}$, câte una pentru fiecare maşină fixată M. (Fiecare şir din K_M începe la fel, cu $\rho(M)$. Nu rezultă imediat din faptul că limbajul K_0 nu este decidabil Turing că anumite submulţimi K_M nu sunt decidabile Turing. În general, este posibil ca un limbaj care nu este decidabil Turing să fie reuniunea unor limbaje decidabile Turing. De exemplu, orice limbaj ce constă dintr-un singur şir este decidabil Turing; el este decis de maşina Turing care verifică dacă intrarea sa este acel şir particular şi se opreşte cu (Y) sau (N) pe bandă. Astfel, orice limbaj este reuniunea unei

mulţimi numărabile de limbaje decidabile Turing. Totuşi, nu există o metodă generală de a combina infinit de multe proceduri de decizie într-una singură.

În cazul nostru particular, ştim că limbajul K_{M_0} nu este decidabil Turing, unde M_0 este maşina Turing care acceptă K_0. Teorema 2.5 rezultă din asta.

2.2. Enumerabilitate, acceptare, decidabilitate Turing

Aşa cum am văzut, orice limbaj decidabil Turing este acceptat Turing dar cele două clase nu coincid. Ce limbaje acceptate Turing sunt decidabile Turing? Există mai multe modalităţi de a răspunde la această întrebare.

Teorema 2.6

Un limbaj este decidabil Turing, dacă şi numai dacă atât el cât şi complementul său sunt acceptate Turing.

Demonstraţie. Dacă L este un limbaj decidabil Turing, atunci L este acceptat Turing (conform Teoremei 2.1). De asemenea complementul său, $\overline{L}$, este decidabil Turing (conform Teoremei 2.2) şi deci acceptat Turing.
Să presupunem că L este acceptat de M_1 şi $\overline{L}$ este acceptat de M_2. Atunci putem construi o maşină Turing M care decide L; M va fi o maşină cu două benzi (care conform Teoremei 1.2 este echivalentă cu o maşină Turing standard). Maşina M începe prin a pune şirul de intrare w pe ambele benzi şi poziţionează capetele la limita dreapta a fiecărui şir. M simulează M_1 şi M_2 în paralel; la fiecare pas al unei operaţii a lui M, un pas al operaţiilor lui M_1 este pus pe prima bandă, şi un pas al operaţiilor lui M_2 este pus pe a 2-a bandă. Cum fie M_1 fie M_2 trebuie să accepte w, dar nu amândouă, M va ajunge în situaţia în care M_1 sau M_2 se va opri. Când se întâmplă, M determina care dintre M_1 sau M_2 accceptă w şi se opreşte cu benzile configurate pentru a indica dacă $w \in L$ sau $w \notin L$. ■

Am definit un limbaj acceptat Turing ca fiind mulţimea şirurilor pentru care o anumită maşină Turing se opreşte. Dar putem caracteriza un limbaj acceptat Turing ca mulţimea şirurilor rămase pe bandă la sfârşitul operaţiilor.

Definiţia 2.1

Fie Σ_0 un alfabet care nu conţine $\#$. Un limbaj $L \subseteq \Sigma_0^*$ este limbajul de ieşire al unei maşini Turing M dacă şi numai dacă L este mulţimea tuturor şirurilor $w \in \Sigma_0^*$ astfel încât, pentru un anumit şir $u \in \Sigma^*$, $(s, \#u\underline{\#}) \vdash_M^* (h, \#w\underline{\#})$, unde s este starea iniţială a maşinii M.

Cu alte cuvinte, w este în limbajul de ieşire a lui M, dacă există un anumit şir de intrare pentru care M se opreşte, lăsând w pe bandă în modul specificat pentru maşina Turing care calculează funcţii.

Teorema 2.7

Un limbaj este acceptat Turing dacă şi numai dacă este limbajul de ieşire al unei maşini Turing.

Demonstraţie. Mai întâi vom demonstra că dacă L este limbajul de ieşire al unei maşini Turing M, atunci L este acceptat Turing. Din teorema de echivalență a maşinii Turing deterministe şi a celei nedeterministe este suficient să demonstrăm că L este acceptat de o maşină Turing nedeterministă. O maşină Turing nedeterministă M' poate fi proiectată pentru a accepta L astfel: pentru un şir de intrare w, M' scrie nedeterminist şirul u pe bandă, salvând w pe a doua pistă a benzii sale. Apoi M' simulează maşina M. La sfârşitul operaţiilor, M' compară şirul pe care M l-ar fi lăsat pe bandă cu w. Dacă sunt identice atunci M' acceptă, altfel M' intră într-un ciclu infinit. Astfel M' acceptă w doar în cazul în care există un anumit şir u din care şirul w ar fi putut fi calculat.
Implicația inversă: dacă L este mulţimea şirurilor de intrare acceptate de o maşină Turing M, atunci putem modifica M astfel încât la acceptarea unui şir de intrare se opreşte cu acel şir pe bandă (ceea ce implică păstrarea intrării în cursul operaţiilor). Mulţimea şirurilor

rămase pe bandă la sfârşitul operaţiilor este aceeaşi cu mulţimea şirurilor acceptate de M adică L. ■

Un alt mod în care mașina Turing poate fi utilizată pentru a specifica un limbaj este prin generarea membrilor săi.

Definiţia 2.2

O mașină Turing enumeră limbajul L dacă şi numai dacă pentru o anumită stare fixată q a lui M,

$$L = \{ w\text{: pentru un şir } u,\ (s, \#) \vdash_{M}^{*} (q, \#w\underline{\#}u) \}$$

unde s este starea iniţială a lui M.
Un limbaj este enumerabil Turing dacă şi numai dacă este enumerat de o anumită mașină Turing.

M enumeră L pornind cu o bandă vidă şi calculând, trecând periodic printr-o stare specială q (nu este starea de oprire). Intrarea în stare q semnalează că şirul rămas la stânga capului este un membru al lui L; M poate ieşi din q şi reintra mai târziu cu un alt membru din L la stânga capului. Șirul u aflat la dreapta capului poate fi utilizat pentru a păstra informaţie despre reluarea operaţiilor după işirea din starea q.

Teorema 2.8

Un limbaj este acceptat Turing dacă şi numai dacă este enumerabil Turing.

Demonstraţie. Să presupunem că L este acceptat de o anumită mașină Turing M . Atunci putem construi o mașină Turing M' care, în loc să primească un şir ca intrare, porneşte cu o bandă vidă, generează sistematic (de exemplu, în ordine lexicografică) toate şirurile peste alfabetul lui L, şi efectuează asupra fiecăruia operaţiile pe care le-ar fi făcut M. Din păcate, modul evident prin care acest scop poate fi atins nu funcţionează: M' s-ar putea să nu termine operaţiile pe un şir înainte de a începe să-l prelucreze pe următorul, pentru că se poate agăţa pentru un anumit şir pentru care operaţiile lui M nu se termină, deşi există alte şiruri pe care M le acceptă, dar care nu au fost încă generate.

În loc să încerce să termine operaţiile pentru un şir atunci când este generat, M' efectuează următoarea secvenţă de operaţii:

(1) efectuează un pas din operaţiile lui M asupra primului şir peste alfabetul lui M
(2) efectuează doi paşi din operaţiile lui M pe fiecare dintre cele două şiruri generate (primele două)
(3) efectuează trei paşi din operaţiile lui M pe fiecare dintre primele trei şiruri

.
.
.

De fiecare dată când M' descoperă că M ar fi acceptat un şir, M' se opreşte în starea q pentru a semnala acceptarea. Un anumit mecanism trebuie utilizat pentru ca M' să ştie care dintre fazele importante ale operaţiilor sale se desfăşoară şi cât de tare a avansat.

Implicația inversă: dacă M enumeră L, atunci putem modifica M să accepte L. Reconstruim M astfel încât să salveze orice intrare primită înainte să înceapă procesul de enumerare. De fiecare dată când M ar intra în starea q, maşina modificată compară conţinutul curent al benzii cu şirul de intrare salvat. Dacă se potrivesc, şirul de intrare este acceptat, altfel procesul de enumerare continuă. Noua maşină acceptă exact limbajul enumerat de M. ■

Modul în care o mașină Turing enumeră un limbaj este reminiscent modului în care o gramatică generează un limbaj. Vom termina caracterizarea limbajelor acceptate Turing, arătând că această similaritate nu este un accident.

Teoremă 2.9

Un limbaj este acceptat Turing dacă şi numai dacă este generat de o gramatică.

Demonstraţie. Să presupunem că L este generat de o gramatică G. Atunci putem proiecta o maşină Turing M care acceptă L. Fie un şir de intrare w, M salvează w şi apoi sistematic produce toate derivările posibile din gramatică. M poate produce toate şirurile peste acest alfabet în ordine lexicografică şi poate determina care dintre ele reprezintă derivări. Ori de câte ori M produce un şir care nu

reprezintă o derivare compară ultimul şir din derivare cu intrarea w şi se opreşte dacă sunt egale; altfel continuă să genereze lexicografic alte derivări.

Implicaţia inversă: să presupunem că $L \subseteq \Sigma_0^*$ este acceptat Turing. Atunci conform Teoremei 2.7 este limbajul de ieşire al unei maşini Turing $M = (K, \Sigma, \delta, s)$.

$$L = \{w: \text{pentru } u \in \Sigma_0^*, (s, \#u\underline{\#}) \vdash_M^* (h, \#w\underline{\#})\}$$

Fie gramatica G_0 construită astfel încât derivările din gramatică să simuleze tranziţiile maşinii Turing (Lema 2.1). G_0 are proprietatea:

$$(s, \#u\underline{\#}) \vdash_M^* (h, \#w\underline{\#})$$

dacă şi numai dacă

$$[\#us\#] \overset{*}{\Rightarrow}_{G0} [\#wh\#]$$

Pentru a obţine gramatica G, trebuie adăugate următoarele reguli la gamatica G_0.

1. Reguli care permit începerea derivărilor şi generarea şirului de intrare:

   $S \rightarrow [\#S_1\#]$

   $S_1 \rightarrow aS_1; \forall a \in \Sigma_0$

   $S_1 \rightarrow s$

   Neterminalele S şi S_1 sunt nou introduse; S este simbolul de start al lui G. Utilizând doar aceste reguli, poate fi generat orice şir de forma $[\#us\#]$, $u \in \Sigma_0^*$.
2. Reguli pentru a termina derivările şi a şterge simbolurile de sfârşit:

   $h\#] \rightarrow X$

   $aX \rightarrow Xa, \forall a \in \Sigma_0$

   $[\#X \rightarrow e$

unde X este un nou neterminal. Pornind cu un şir de forma $[\#wh\#]$, prima regulă generează $[\#wX$, apoi prin $|w|$ aplicări ale regulilor de tipul al doilea rezultă $[\#Xw$; şi în final, aplicând ultima regulă, w. ■

***Obținerea lui* G_0**

Lema 2.1

Fie $M = (K, \Sigma, \delta, s)$. Atunci există o gramatică G astfel încât pentru orice configuraţii (q, u, a, v) şi (q', u', a', v') ale lui M,

$$(q, u, a, v) \vdash_M^* (q', u', a', v')$$

dacă şi numai dacă:

$$[uqav] \Rightarrow_G^* [u'q'a'v']$$

Unde $[$ şi $]$ sunt simboluri noi care nu există în Σ sau K; presupunem de asemenea că mulțimile K şi Σ sunt disjuncte.

Demonstraţie. Ideea este de a reprezenta configuraţia (q, u, a, v) prin şirul $[uqav]$; poziţia stării în şir serveşte şi ca indicaţie a poziţiei capului. Acestă reprezentare permite ca partea stânga a fiecărei reguli să conţină ca subşir o stare şi simbolul scanat; numai starea, simbolul scanat şi poziţia capului trebuie schimbate la trecerea de la o configuraţie la următoarea.

Simbolurile $[$ şi $]$ apar doar la capetele reprezentării unei configuraţii. Simbolul $]$ este utilizat pentru a trata banda vidă, iar simbolul $[$ este introdus pentru legilibitate.

Formal, $G = (V, \Sigma, R, S)$ (simbolul de start nu are niciun rol în condiţia pe care G trebuie să o îndeplinească).

$$V = K \cup \Sigma \cup \{[,], h, S\}$$

iar R conţine regulile:

(a) $\forall q \in K$, $a \in \Sigma$, dacă $\delta(q, a) = (p, b)$, $p \in K \cup \{h\}$, $b \in \Sigma$ atunci R conţine regula

$$qa \to ph$$

(b) $\forall q \in K$, $a \in \Sigma$, dacă $\delta(q, a) = (p, R)$, $p \in K \cup \{h\}$, R conţine regula

$$qab \to aph, \text{ pentru } \forall\, b \in \Sigma, \text{ şi}$$

$$qa] \to ap\#$$

(c) $\forall q \in K$, $a \in \Sigma$, dacă $\delta(q, a) = (p, L)$, $p \in K \cup \{h\}$, R conţine regula

$$bqac \rightarrow pbac$$

$\forall\ b \in \Sigma$ şi $c \in \Sigma \cup \{]\}$, astfel încât $a \neq \#$ sau $c \neq]$, şi

$$bq\#] \rightarrow pb],$$

$\forall\ b \in \Sigma$, dacă $a = \#$

3. COMPLEXITATE

În capitolul precedent am arătat că există probleme care nu pot fi rezolvate prin algoritmi. Prin urmare putem clasifica problemele în două categorii: probleme care admit, respectiv care nu admit soluții algoritmice. Având în vedere dezvoltarea tehnologică din ultimii ani, ne-am aştepta că toate problemele din prima categorie să poată fi rezolvate cu uşurinţă de calculatoarele actuale. Din păcate, practica arată că deşi teoretic rezolvabile, anumite probleme nu pot fi rezolvate practic datorită timpului foarte mare necesitat.

De exemplu, să presupunem că trebuie să planificăm vizitele pe care un comis voiajor trebuie să le facă în 10 oraşe. Se dă harta cu cele 10 oraşe şi distanţele lor şi trebuie să se obţină itinerariul care minimizează distanța parcursă. Din punct de vedere teoretic problema este rezolvabilă. Dacă sunt n oraşe, numărul itinerariilor posibile este finit, respectiv $(n-1)!$ Deci trebuie proiectat un algoritm care determină itinerariul minim.

Pentru problema celor 10 oraşe, trebuie examinate $9! = 362980$ itinerarii, un număr încă calculabil. Dar dacă trebuie vizitate 30 de oraşe? Numărul intinerariilor este foarte mare: $29! >> 10^{30}$. Chiar dacă s-ar putea examina un miliard de tururi pe secundă (ceea ce este mult peste capabilităţile actuale ale calculatoarelor), timpul necesar pentru a termina este mai mare decât un miliard de vieţi omeneşti.

Deci, faptul că o problemă este teoretic rezolvabilă nu implică că ea poate fi rezolvată practic. Desigur, clasificarea în probleme rezolvabile şi nerezolvabile trebuie rafinată. Problemele rezolvabile trebuie rafinate în concordanţă cu complexitatea lor.

Avem deja o astfel de clasificare. Ştim că un limbaj decidabil Turing poate fi regulat, independent de context dar nu poate fi neregulat sau neindependent de context. În fapt, ştim că există limbaje independente de context care nu sunt independente de context deterministe. Deci avem o ierarhie de limbaje decidabile Turing: decidabile Turing – independente de contex – independente de contex deterministe – regulate. Această ierarhie reflectă o distincţie calitativă între automatele corespunzătoare.

De exemplu, un automat cu stivă este în esenţă o maşină Turing cu două benzi cu restricţii asupra mişcării capetelor şi a scrierii; un automat finit este un automat cu stivă care nu utilizează stiva.

Scopul nostru în acest capitol, este de a stabili o altă clasificare cantitativă a limbajelor decidabile, considerând maşini Turing care nu sunt restricţionate structural, ci în cantitatea de efort pe care o pot consuma în timpul operaţiilor efectuate asupra unui şir de intrare.

O măsură naturală a cantităţii de efort depuse de o maşină Turing este numărul de paşi utilizaţi pentru a ajunge la o concluzie referitoare la intrare.

Definiţia 3.1

Fie M este o maşină Turing şi $t \in \mathbb{N}$, definim relaţia $\vdash_M^t$ între configuraţiile lui M astfel:

$C \vdash_M^0 C'$ dacă şi numai dacă $C = C'$

$C \vdash_M^{t+1} C'$ dacă şi numai dacă există C'' astfel încât $C \vdash_M^t C''$ şi $C'' \vdash_M^t C'$

Adică,

$C \vdash_M^t C'$ dacă şi numai dacă $C \vdash_M^* C'$ printr-un calcul în exact t paşi.

Definiţia 3.2

Fie T o funcţie, $T: \mathbb{N} \to \mathbb{N}$. Fie $L \subseteq \Sigma_0^*$ un limbaj şi $M = (K, \Sigma, \delta, s)$ o maşină Turing cu k benzi şi $\Sigma_0 \subseteq \Sigma$. Spunem că M decide L în timpul T, dacă:

pentru $w \in L$,

$(s, \#w\underline{\#}, \underline{\#}, \ldots, \underline{\#}) \vdash_M^t (h, \#\textcircled{Y}\underline{\#}, \underline{\#}, \ldots, \underline{\#})$, $t \leq T(|w|)$

şi pentru $w \notin L$,

$(s, \#w\underline{\#}, \underline{\#}, \ldots, \underline{\#}) \vdash_M^t (h, \#\textcircled{N}\underline{\#}, \underline{\#}, \ldots, \underline{\#})$, $t \leq T(|w|)$.

Spunem că L este decidabil în timpul T, dacă există $k > 0$ şi o maşină Turing cu k benzi care decide L în timpul T. Clasa limbajelor decidabile în timpul T este notată TIME(T).

Am limitat numărul de paşi ai unei maşini Turing nu printr-o constantă absolută, ci în termenii unei funcţii dependente de lungimea intrării.

Având în vedere că o maşină Turing trebuie să şteargă intrarea şi să scrie Ⓨ sau Ⓝ înainte de a se opri pentru intrarea w trebuie să efectueze $|w| + 1$ scrieri şi $|w| + 3$ mutări ale capului, deci nicio funcţie T astfel încât $T(n) < 2n + 4$, $n \geq 4$ nu este o limită de timp utilă.

Exemplu 3.1

Fie $L = \{w \in \{a, b\}^* : w$ nu conţine nicio apariţie a şirului aa$\}$

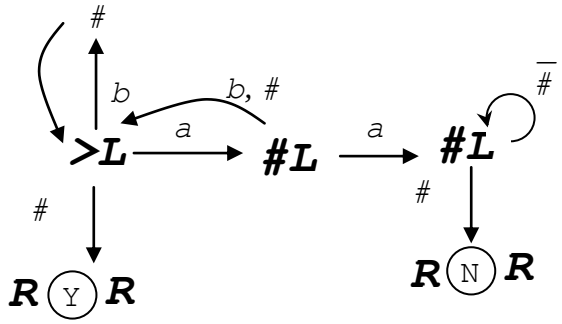


Figura 3-1

Maşina Turing parcurge şirul de la dreapta la stânga, verificând existenţa şirului aa, ştergând în acelaşi timp intrarea. În total, execută $|w| + 1$ scrieri şi $|w| + 3$ deplasări. M decide L în T, $T(n) = 2n + 4$; L $\in$ TIME (T).

Exemplu 3.2

Fie maşina Turing care decide limbajul $L = \{w \in \{a, b, c\}^* : w$ are acelaşi număr de simboluri a, b, c$\}$.

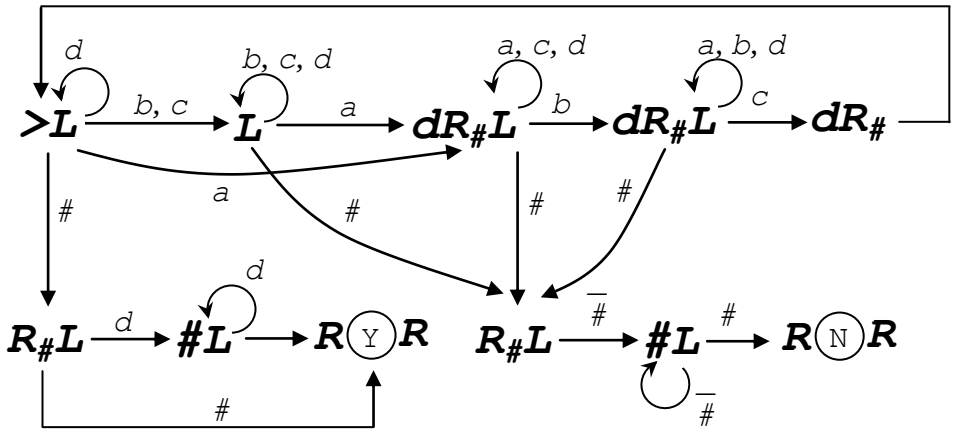


Figura 3-2

M funcţionează în etape. La fiecare etapă începe prin a scana intrarea de la dreapta la stângă, până când întâlneşte un simbol a, care este înlocuit cu simbolul d; apoi M se întoarce înapoi la capătul dreapta, scanează din nou banda, până când întâlneşte un simbol b pe care îl înlocuieşte d. Operaţiile sunt repetate pentru simbolul c, terminând astfel o etapă. Dacă M într-o anumită etapă caută un simbol a dar găseşte numai simboluri d, şterge banda şi se opreşte cu #Y#; în cazul unei căutări nereuşite M şterge banda şi se opreşte cu #N#.

Să numărăm paşii utilizaţi de M pentru o intrare de lungime n. La fiecare etapă, M scanează banda de trei ori; la fiecare scanare, M examinează cel mult n simboluri, înlocuieşte cel mult unul, şi repoziţionează capul la limita dreapta a benzii după cel mult n paşi – în total $(2n + 1)$ paşi. Prin urmare, fiecare etapă constă din cel mult $3(2n + 1)$ paşi. Câte etape sunt? Există cel mult $[n/3] + 1$, pentru că există cel mult $[n/3] + 1$ simboluri a, b sau c. La sfârşit M şterge banda şi scrie Y sau N în $2n + 4$ paşi. Aceasta face ca numărul total de paşi să fie un număr mai mic decât $([n/3] + 1) \infty 3(2n + 1) + 2n + 4$. Pentru a simplifica expresia, să observăm, că pentru orice n, $[n/3] \leq (n + 2) / 3$, deci $T(n) = 2n^2 + 13n + 9$. Cum M decide L în T, $L \in \text{TIME}(2n^2 + 13n + 9)$.

Aproximarea este însă pesimistă pentru anumite intrări. De exemplu pentru b^n, M execută doar $4n + 5$ paşi înainte de a se opri cu #N# pentru că

detectează imediat că nu există a. Deci, limita $T(n)$ reflectă comportarea lui M în cazul cel mai defavorabil pentru un şir de lungime n.

Deşi calculul complexităţii $(O(n))$ pentru o maşină Turing poate fi netrivial, proiectarea unei maşini Turing care să decidă un limbaj L într-un timp dat T, $L \in TIME(T)$, este mai interesant. Scopul calculului de complexitate este de a alege dintre mai multe maşini Turing, pentru a decide un limbaj, pe cea ale cărei operaţii se termină în T paşi, unde T este dat sau dacă nu este posibil, să demonstrăm riguros această imposibilitate.

Exemplu 3.3

Fie L limbajul din exemplul anterior. Există o maşină Turing cu k-benzi, M′ $(k \geq 1)$ astfel încât M′ să decidă L în cazul cel mai defavorabil în mai puţini paşi decât M? Adică $L \in TIME(T')$, unde $T' < T$?

Construim o maşină Turing M′ cu patru benzi. M′ scanează intrarea de la dreapta la stânga, ştergând simbolurile de pe bandă, şi în acelaşi timp scrie un simbol I pe a doua bandă pentru fiecare a pe care îl întâlneşte, un I pe a treia bandă pentru fiecare b întâlnit, şi un I pe a patra bandă pentru fiecare c întâlnit. Apoi M′ compară numărul de simboluri I pe fiecare dintre benzile a doua, a treia şi a patra, mutând capetele benzilor, la stânga, simultan. Dacă pe fiecare bandă simbolul curent este I, continuă să-şi deplaseze capetele, ştergând simbolurile I de pe toate cele trei benzi. Dacă întâlneşte un simbol # (cel mai din stânga) pe toate cele trei benzi, atunci se opreşte cu #Ⓨ# pe prima bandă, altfel se opreşte cu #Ⓝ# după ce şterge simbolurile I rămase.

Vom utiliza următoarele notaţii:

- $[A^{(i)}, B^{(j)}] \Rightarrow$ acţiuni pe benzile i şi j executate simultan
- $A^{(i,j,k)} = [A^{(i)}, A^{(j)}, A^{(k)}]$; de exemplu $[\#^{(1)}, I^{(4)}]$, scrie # pe banda 1 şi I pe banda 4 simultan.
- $L^{(2,3,4)} \Rightarrow$ mută capetele pe benzile 2, 3, 4 la stânga în acelaşi timp.

Intuitiv, M' este mai eficient decât M; să estimăm cazul cel mai defavorabil pentru un şir de intrare de lungime n. Vom analiza trei cazuri corespunzătoare celor trei submaşini (1), (2), (3) .

1. Citirea şi ştergerea intrării se face în $3n + 1$ paşi.
2. Fie p numărul de apariţii în şirul de intrare a unui simbol din $\{a, b, c\}$ care apare de cele mai puţine ori. Atunci compararea

numărului de simboluri I pe cele trei benzi se face în $2p + 1$ paşi.

3. Ştergerea simbolurilor I rămase şi scrierea Ⓨ, Ⓝ se face în cel mult $2(n - 3p) + 2$ paşi.

Prin urmare M' execută $3n + 1 + 2p + 1 + 2(n - 3p) + 2 \leq 5n + 4$ pentru orice şir de lungime n. Deci $L \in \mathrm{TIME}(5n + 4)$.

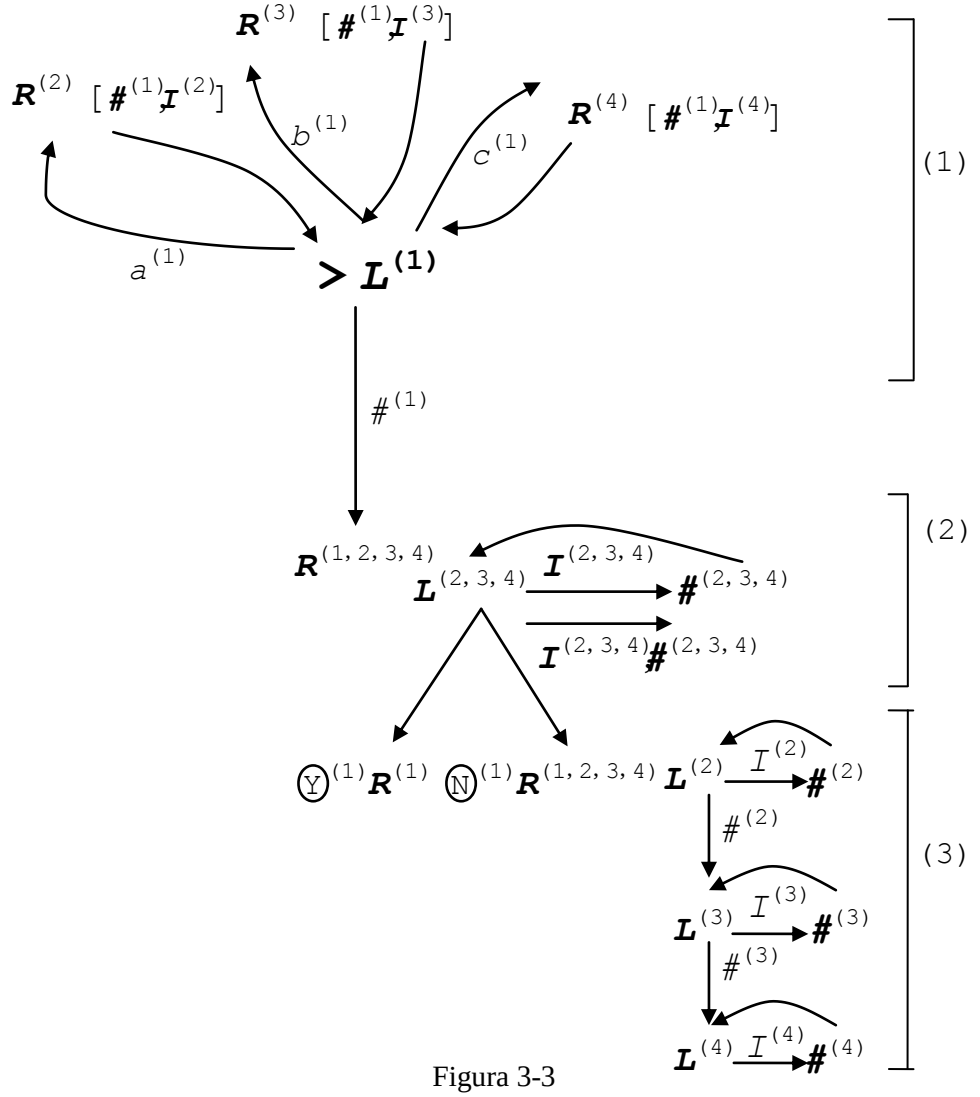


Figura 3-3

Poate fi L decis mai repede? Respectiv $L \in \mathrm{TIME}(cn + d)$, pentru $c < 5$?

Constanta c nu poate fi mai mică decât 2, pentru că sunt necesari cel puţin $2n + 4$ paşi pentru orice mașină Turing, pentru a respecta convenţiile de intrare / ieşire impuse.

Următoarea teoremă demonstrează că c poate fi oricât de apropiat de 2. Mai important, teorema afirmă că întrebări referitoare la coeficienţi constanţi, cum ar fi c, sunt lipsite de importanţă, pentru că aceste constante pot fi întotdeauna micșorate.

Teorema 3.1

Presupunem $L \in \mathrm{TIME}(T)$, şi fie ϵ orice număr real, pozitiv (oricât de mic). Atunci $L \in \mathrm{TIME}(T')$ unde pentru $\forall\, n \in \mathbb{N}$, $T'(n) = 2n + 18 + \lceil \epsilon \infty T(n) \rceil$

Demonstraţie. Fie $M = (K, \Sigma, \delta, s)$ o mașină Turing cu k -benzi care decide L în timpul T. Vom construi o mașină Turing cu M' benzi, $M' = (K', \Sigma', \delta', s')$ care acceptă L în T' unde $T'(n) = 2n + 18 + \lceil \epsilon \times T(n) \rceil$. Dacă $k = 1$ atunci $k' = 2$, altfel $k' = k$.
Operaţiile lui M' au 4 etape. În prima etapă, M' copiază intrarea de pe prima bandă pe a doua, în acest timp comprimând şi inversând intrarea. În a doua etapă, M' copiază versiunea comprimată a intrării înapoi pe prima bandă, reinversând şirul dar păstrând compresia. În a treia etapă, M' simulează M; compresia intrării face posibilă simularea mai multor paşi ai lui M cu mai puţini paşi a lui M', şi acesta este locul unde îmbunătăţirea vitezei se obţine. În a patra etapă M' efectuează paşii necesari pentru a lăsa benzile la fel cu ale lui M. (cu acelaşi conţinut ca în cazul în care M ar fi făcut operaţiile).
M' poate fi descris ca o combinaţie a maşinilor $M_1M_2M_3M_4$; vom descrie aceste maşini pe rând. Pentru a simplifica descrierea, vom presupune că numărul benzilor lui M, k, este 1.
Maşina M_1 porneşte cu şirul de intrare w pe prima bandă. Deplasează capul pe prima bandă la stânga, ştergând w, în acelaşi timp producând o versiune comprimată a intrării $\mathcal{K}(\#w)$ şi scriind intrarea inversată pe a doua bandă. Pentru un anumit întreg pozitiv m (care depinde de ϵ, va fi fixat ulterior în demonstraţie), $\mathcal{K}(\#w)$ este mai scurt decât $\#w$ cu un factor proporţional cu m, respectiv

$|\mathcal{K}(\#w)| = \lceil (1 + |w| + 1) / m \rceil$. Această compresie este obţinută codificând fiecare m–tuplu de simboluri ale lui $\#w$ ca un singur simbol din $\mathcal{K}(\#w)$ (în cazul în care $|w| + 1$ nu este un multiplu exact al lui m, trebuie introduse cel mult $m-1$ simboluri $\#$ în primul simbol din $\mathcal{K}(\#w)$. De exemplu, dacă $w = abbabaa$, $m = 3$, $K(\#w) = (\#, \#, a)(b, b, a)\ (b, a, a)$, unde tripletele sunt simboluri în alfabetul lui M'.

La sfârşitul primei etape, a doua bandă va conţine inversul compresiei $\mathcal{K}(\#w)$, respectiv $(b, a, a)\ (b, b, a)\ (\#, \#, a)$

Formal, etapa de compresie poate fi descrisă astfel. Fie $w \in \Sigma^*$. Dacă $|w| \leq m$, $w = \sigma_1 \ldots \sigma_j$, $j \leq m$, $\sigma_i \in \Sigma$, atunci $\mathcal{K}(w) = (\#, \#, ..., \#, \sigma_1, ..., \sigma_j)$ ($m-j$ apariţii ale simbolului $\#$). Altfel dacă $|w| > m + 1$, $w = w'\sigma_1 \ldots \sigma_m$, $w \in \Sigma^+$ şi $\sigma_1, ..., \sigma_j \in \Sigma$, atunci $\mathcal{K}(w) = \mathcal{K}(w')\tau$, unde $\tau = (\sigma_1, ..., \sigma_m)$. Scopul lui M_1 este să transforme $(\#w\underline{\#}, \underline{\#})$ în $(\underline{\#}, \#\ \mathcal{K}(\#w)^R\underline{\#})$. Este posibilă construcţia lui M_1 astfel încât să facă acest lucru în $2(w) + 1$ paşi.

Maşina M_2 copiază $\mathcal{K}(\#w)$ pe prima bandă în timp ce şterge $\mathcal{K}(\#w)^R$ de pe a doua bandă, adică transformă $(\underline{\#}, \#\mathcal{K}(\#w)^R\underline{\#})$ în $(\#\mathcal{K}(\#w)\underline{\#}, \underline{\#})$. M_2 utilizează doar $2|\mathcal{K}(\#w)^R| + 1$ paşi, respectiv $2\lceil (w)+1/m) \rceil + 1$ paşi. M_2 lasă banda în forma standard.

Maşina M_3 simulează în mod repetat m paşi ai lui M prin 12 paşi proprii. Prima bandă a lui M_3 va conţine – după 12 paşi – $K(x)$, unde x este conţinutul primei benzi a lui M după m paşi. Starea lui M_3 după 12 paşi este de forma (q, j) unde $q \in K$ este starea în care M ar fi ajuns după m paşi, şi j, $1 < j < m$, indică simbolul din m-tuplul scanat de M_3 care corespunde simbolului scanat de M. De exemplu, dacă $m = 3$ şi $7 \infty m = 21$ paşi, M ar fi fost în configuraţia $(q, aab\underline{b}aba)$, atunci M_3 după ce a pornit compresia va fi în configuraţia $((q, b), (\#, \#, a)(a, b, b)(a, b, a))$ după $7 \infty 12 = 84$ paşi.

Simularea următorilor m paşi ai lui M se desfăşoară astfel: mai întâi capul lui M_3 se mută cu o poziţie la stânga, două poziţii la dreapta, o poziţie la stânga, oprindu-se în poziţia din care a pornit. rezultatul este că M_3 a codificat în starea sa cele trei m–tupluri aflate în poziţia curentă, respectiv stânga, dreapta capului său; numim concatenarea acestor m–tupluri şir activ. Acum M_3 are toată informaţia necesară pentru a executa următorii m paşi ai lui M (M nu poate în m paşi să

citească / scrie poziţii în afara şirului activ). Astfel, cunoscând şirul activ şi stările lui M, M_3 poate spune exact care vor fi următorii m paşi ai lui M. Această predicţie poate fi făcută într-un pas al lui M_3, cu ajutorul unei tabele finite, care poate fi construită în avans în funcţia δ_3.Acum M_3 efectuează încă 8 paşi – 3 paşi de scriere, 5 de deplasare – pentru a înregistra noile m–tupluri care trebuie să înlocuiască şirul activ, şi în final să repoziţioneze capul pentru următorul bloc de 12 paşi.

Formal, stările lui M_3 sunt:

$K_3 = K \times \{1, 2 \ldots, m\} \cup K \times \{1, 2 \ldots, m\} \times \Sigma^{3m} \times \{1, 2 \ldots, 11\}$

Starea (q, j, x, p), $1 \le p \le m$, $x \in \Sigma^{3m}$, este starea în care ajunge starea (q, j) după p din cei 12 paşi implicaţi în simularea celor m paşi ai lui M; x este subșirul cunoscut din șirul activ, iar simbolurile # reprezintă partea necunoscută din șir.

Cum M_3 nu consultă / actualizează a doua bandă, vom scrie $\delta_3(q, \sigma) = (q', \sigma')$ în loc de $\delta_3(q, \sigma, \#) = (q', \sigma', \#)$. M_3 are următoarele tranziţii pentru $q \in K$, $1 \le j \le m$, $x, y, z \in \Sigma^m$:

$$\delta_3((q, j), x) = ((q, j, \#^{3m}, 1), L)$$

$$\delta_3((q, j, \#^{3m}, 1), x) = ((q, j, x\#^{2m}, 2), R)$$

$$\delta_3((q, j, x\#^{2m}, 2), y) = ((q, j, xy\#^{m}, 3), R)$$

$$\delta_3((q, j, xy\#^{m}, 3), z) = ((q, j, xyz, 4), L)$$

Fie $xyz = u\sigma v$, $|u| = m + j - 1$, presupunem că $(q, u\underline{\sigma}v) \vdash_M^m (q', u'\underline{\sigma'}v')$ (sau $(q, u\underline{\sigma}v) \vdash_M^t (q', u'\underline{\sigma'}v')$, unde $q' = h$ şi $t < m$) şi $u'\underline{\sigma'}v' = x'y'z'$, unde $x', y', z' \in \Sigma^m$. Atunci

$$\delta_3((q, j, xyz, 4), y) = ((q, j, xyz, 5), L)$$

$$\delta_3((q, j, xyz, 5), x) = ((q, j, xyz, 6), x')$$

$$\delta_3((q, j, xyz, 6), x') = ((q, j, xyz, 7), R)$$

$$\delta_3((q, j, xyz, 7), y) = ((q, j, xyz, 8), y')$$

$$\delta_3((q, j, xyz, 8), y') = ((q, j, xyz, 9), R)$$

$$\delta_3((q, j, xyz, 9), z) = ((q, j, xyz, 10), z')$$

$$\delta_3((q, j, xyz, 10), z') = ((q, j, xyz, 11), L)$$

$$\delta_3((q, j, xyz, 11), y') = \begin{cases} ((q', |u'| + 1), L), & |u'| + 1 \le m \\ ((q', |u'| + 1 - m), y'), & m < |u'| + 1 \le 2m \end{cases}$$

$((q', |u'| + 1 - 2m), R), 2m < |u'| + 1$

În continuare trebuie să ţinem cont de ceea ce se întâmplă când M_3 înâlneşte un nou simbol #. Putem forţa M_3 să trateze un simbol # ca un *m*-tuplu de simboluri #, astfel încât atunci când M_3 întâlneşte un #, simularea să continue fără întârzieri suplimentare pentru compresie. Acelaşi efect se obţine modificând funcţia δ_3 definită anterior înlocuind $\#^m$ cu # oriunde apare, ca argument sau valoare.
La sfârşit, M_4 trebuie să convertească rezultatul lăsat de M_3, când se opreşte, în şirul pe care M l-ar fi lăsat pe bandă. M_4 trebuie să transforme $\#(\#, \underline{Y}, \#, \ldots, \#)$ în $\#\underline{Y}\#$ şi $\#(\#, \underline{N}, \#, \ldots, \#)$ în $\#\underline{N}\#$. Deci M_4 efectuează doar doi paşi. Câţi paşi, în cazul cel mai defavorabil, necesită $M' = M_1M_2M_3M_4$ pentru w, $|w| = n$? M_1 face $2n + 1$ paşi, M_2 - $2\lceil (n + 1) / m \rceil + 1$ paşi, M_4 – doi paşi. În cazul cel mai defavorabil M_3 face $12\lceil T(n)/m \rceil$ paşi, pentru că m paşi ai lui M sunt simulaţi prin 12 paşi ai lui M_3 şi M nu face mai mult de $T(n)$ paşi. Deci în cazul cel mai defavorabil,

$$12\left\lceil \frac{T(n)}{m} \right\rceil + 2\left\lceil \frac{n+1}{m} \right\rceil + 2n + 4 \text{ paşi}$$

Pentru $\forall\, r, s \in \mathbb{N} - \{0\}$,

$$\left\lceil \frac{r}{s} \right\rceil \leq \frac{r + s - 1}{s},$$

numărul de paşi este cel mult:

$$\frac{12T(n) + 12m - 12 + 2n + 2 + 2m - 2}{m} + 2n + 4;$$

cum $T(n) \geq 2n + 4$, acest număr nu este mai mare decât

$$\frac{13T(n)}{m} + 2n + 18;$$

În final, putem alege un m satisfăcător. Dacă $m = \lceil 13/\epsilon \rceil$, atunci M' decide L în T'

$$T'(n) = 2n + 18 + \lceil \varepsilon \times T(n) \rceil, \forall n \geq 0$$

3.1. Rata de creştere a funcţiilor

Teorema precedentă afirmă că atunci când se lucrează cu limita superioară T a unei maşini Turing, cel mai important lucru este rata de creştere a funcţiei T şi nu detalii de tipul coeficienţilor constanţi sau termeni de ordin inferior.

Definiţia 3.3

Fie f, g: N $\rightarrow$ N, $f = O(g)$ dacă şi numai dacă există $c > 0$ şi $n_0 \in$ N astfel încât $f(n) \leq c \infty g(n), n \geq n_0$.

De exemplu:

$50n^2 + 14n + 33 = O(n^2)$

$n \geq 1, n^2 \geq n, n^2 \geq 1$

$50n^2 + 14n + 33 \leq 97n^2, n \geq 1$

$c = 97, n_0 = 1.$

Lema 3.1

Fie $\leq$ relaţia definită pe mulţimea funcţiilor definite pe mulţimea numerelor naturale. $f \leq g$ dacă şi numai dacă $f = O(g)$. Atunci relaţia $\leq$ este reflexivă şi tranzitivă.

Demonstraţie. Relaţia $\leq$ este reflexivă, pentru că $f = O(f), \forall f, c = 1, n_0 = 0$. Este şi tranzitivă. Presupunem că $f = O(g)$ şi $g = O(h)$ atunci $\exists c, c' > 0$ şi $n_0, n_0' \in$ N astfel încât

$f(n) \leq c \infty g(n), n \geq n_0$

$g(n) \leq c' \infty h(n), n \geq n_0'$

$f(n) \leq c \infty c' \infty h(n), n \geq \max(n_0, n_0')$ şi deci $f = O(h)$

Să definim relaţia $\equiv$ astfel:

$f \equiv g$ dacă şi numai dacă $f = O(g)$ şi $g = O(f)$

Cum $\leq$ este reflexivă şi tranzitivă, $\equiv$ este relaţia de echivalenţă ■

Definiţia 3.4

Clasa de echivalenţă a funcţiei f: N $\rightarrow$ N în raport cu relația de echivalență ≡ este numită rata de creştere a funcţiei f.

De exemplu,

$50n^2 + 14n + 33 = O(n^2)$

şi $n^2 = O(50n^2 + 14n + 33) \Rightarrow n^2 \equiv 50n^2 + 14n + 33$

adică n^2 şi $50n^2 + 14n + 33$ au aceeași rată de creştere

Lema 3.2

Fie $f(n) = \sum_{j=0}^{d} a_j n^j$ un polinom de grad d. Atunci:

(a) $f \equiv n^d$;

(b) $f = O(n^{d+1})$ şi $f \not\equiv n^{d+1}$,

Demonstraţie.

(a) Rezultă imediat că

$$f(n) \leq (\sum_{j=0}^{d} b_j) \times n^d,\ b_j = \max(a_j, 0)$$

Deci $f(n) = O(n^d)$, unde

$$c = (\sum_{j=0}^{d} b_j),\ n_0 = 1$$

Fie

$$n_0 = 2 \sum_{j=0}^{d-1} |a_j|$$

este uşor de demonstrat că $n^d \leq 2 \infty f(n)$, $n \geq n_0$

(b) Este evident că $n^d = O(n^{d+1})$

$f = O(n^{d+1})$ pentru că din (a) $f \equiv n^d$.

Pentru a arăta că nu este posibil ca $n^{d+1} = O(n^d)$, să presupunem că $\forall\ n \geq n_0$, $n^{d+1} \leq c \infty n^d$; această inegalitate nu mai este adevărată pentru $n = \max(\lceil c \rceil) + 1, n_0)$ ■

Lema arată că rata de creştere a unui polinom poate fi identificată cu gradul său.

În afară de polinoame, există anumite rate de creştere care deşi nu sunt sub formă polinomială, sunt mărginite superior de polinoame. De exemplu:

$n\lceil \log_2(n+1) \rceil$ şi $n^2\lfloor \mathrm{sqrt}(n) \rfloor$

$n\lfloor \log(n+1) \rfloor = O(n^2)$

$n^2\lfloor \mathrm{sqrt}(n) \rfloor = O(n^3)$

Aceste limite polinomiale ale ratei de creştere sunt interesante prin contrast cu ratele exponenţiale de forma r^n, $r > 1$. Diferenţa este stabilită de următoarea teoremă.

Teorema 3.2

Fie $f(n) = \Sigma_{j=0}^{d} a_j n^j$ şi $r > 1$. Atunci $f = O(r^n)$

Demonstraţie. Este suficient să demonstrăm că $n^d = O(r^n)$.
$\forall\ n > 0$, fie $e_n = r^n$ şi $f_n = n^d$. Atunci $e_{n+1}/e_n = r$,

$$\frac{f_{n+1}}{f_n} = \frac{(n+1)^d}{n^d} = \left(1 + \frac{1}{n}\right)^d$$

$\frac{f_{n+1}}{f_n}$ descreşte cu n şi este mai mică decât r, atunci când

$$\left(1 + \frac{1}{n}\right)^d < r;$$

adică atunci când,

$$n > \frac{1}{r^{1/d} - 1}$$

Fie $n_0 = \lceil 1/(r^{1/d} - 1) \rceil$ şi $c = n_0^d / r^{n0}$. Atunci $\forall\ k \geq 0$,

$$(n_0 + k)^d = n_0^d \left(\frac{n_0 + 1}{n_0}\right)^d \propto \left(\frac{n_0 + 2}{n_0}\right)^d \propto \cdots \propto \left(\frac{n_0 + k}{n_0}\right)^d$$

$$\infty \qquad n_0 \qquad n_0+1 \qquad\qquad n_0+k-1$$

$$= n_0^d \left(1+\frac{1}{n_0}\right)^d \propto \left(1+\frac{1}{n_0+1}\right)^d \infty \ldots \infty 1+\frac{1}{n_0+k-1}\Bigg)^d \leq n_0^d r^k$$

$$\infty$$

$= cr^{n0} r^k = cr^{n0+k}$, $\forall\ n \geq n_0$, $n^d = O(r^n)$.

pentru că:

$$\left(1+\frac{1}{n}\right)^d \leq r,\ n \geq n_0$$

■

Cum pentru orice $d > 0$, n^{d+1} creşte mai repede decât n^d, putem afirma (Teorema 3.1) că orice funcţie exponenţială creşte mai repede decât una polinomială.

Pentru a simplifica notaţia vom nota $\mathrm{TIME}(f)$ pentru

$$\cup \{\mathrm{TIME}(g) : g \equiv f\}$$

pentru a reprezenta clasa limbajelor decidabile într-o limită de timp modelată printr-o funcţie având aceeaşi rată de creştere ca şi f.

3.2. Limite de timp

Teorema 3.3

Fie L un limbaj decis de o maşină Turing M cu bandă infinită în ambele sensuri în timpul T_1. Atunci L este decis de o maşină Turing M_2 în T_2, unde $\forall\ n \in \mathrm{N}$, $T_2(n) = 6T_1(n) + 3n + 8$.

Teorema 3.4

Fie L un limbaj decis în timpul T_1 de o maşină Turing cu k benzi, $M_1 = (K_1, \Sigma_1, \delta_1, s_1)$. Atunci L este decis de o maşină Turing cu o

bandă $M_2 = (K_2, \Sigma_2, \delta_2, s_2)$ în T_2, unde $\forall\ n \in N$, $T_2(n) = 4T_1(n)^2 + (4n + 4k + 3)T_1(n) + 5n + 15$.

Corolar

Dacă L este un limbaj decis în timpul T de o anumită mașină Turing cu k benzi, atunci L este decis în $T' = O(T^2)$ de o mașină Turing cu o bandă.

Teorema 3.5

Dacă L este un limbaj decis în timpul T de o mașină Turing bidimensională, atunci L este decis în $T' = O(T^3)$ de o mașină Turing cu patru benzi.

3.3. Clasele $\mathcal{P}$ și $\mathcal{NP}$

Definiţia 3.5

Definim $\mathcal{P}$ (decidabil în timp polinomial) clasa limbajelor

$$\mathcal{P} = \cup \{TIME(n^d) : d > 0\}$$

unde $TIME(n^d)$ conţine toate limbajele decise de o mașină Turing într-o limită de timp care este un polinom de grad d. Considerând reuniunea peste toate gradele obţinem clasa tuturor limbajelor decise de o mașină Turing în limite polinomiale.

Clasa $\mathcal{P}$ introdusă are un rol important. Definește clasa limbajelor decidabile, din punct de vedere cantitativ. Clasa limbajelor decidabile coincide (Teza lui Church) cu clasa problemelor care pot fi rezolvate algoritmic. $\mathcal{P}$ coincide cu clasa problemelor care pot fi rezolvate practic.

Definiţia 3.6

Fie $T: N \to N$, $L \subseteq \Sigma_0^*$ un limbaj şi $M = (K, \Sigma, \delta, s)$ o mașină Turing nedeterministă. M acceptă L nedeterminist în timpul T dacă:

$\forall\ w \in \Sigma_0^*$, $w \in L$ dacă şi numai dacă $(s, \#w\#) \vdash_M^t (h, v\underline{\sigma}u)$, $v, u \in \Sigma_0^*$, $\sigma \in \Sigma$, $t \le T(|w|)$.

Spunem că L este acceptat nedeterminist în timpul T, dacă există maşina Turing nedeterministă M care acceptă L nedeterminist în timpul T.

Clasa limbajelor acceptate nedeterminist în timpul T este notată cu $\mathrm{NTIME}(T)$.

Notăm $\mathcal{NP} = \cup \{\mathrm{NTIME}(n^d) : d > 0\}$.

Noţiunea de acceptare nedeterministă limitată în timp este analogul cantitativ al acceptării nedeterministe. Spunem că o maşină Turing nedeterministă M acceptă w dacă se opreşte pentru intrarea w; nu acceptă dacă operaţiile sale sunt infinite. În ultimul caz, avem aceeaşi convenţie cu diferenţa că este considerată infinită orice operaţie cu mai mult de $T(|w|)$ paşi pentru intrarea w.

De remarcat o asimetrie fundamentală în definirea calculului nedeterminist limitat de timp. O maşină Turing nedeterministă acceptă şirul w în timpul T dacă şi numai dacă există una sau mai multe operaţii care duc la configuraţia de oprire după $T(|w|)$ sau mai puţini paşi. Nu acceptă dacă operațiile mașinii continuă după mai mult de $T(|w|)$ paşi. În figura 3-4 (a) şirul este acceptat, în (b) nu.

Mulţimea tuturor operaţiilor unei maşini Turing nedeteministe pentru o intrare w poate fi reprezentată ca un arbore. Nodurile reprezintă configurații, arcele paşi. Alegerile nedeterministe sunt reprezentate prin noduri care au mai mult de un arc.

Este imediat faptul că $\mathrm{TIME}(T) < \mathrm{NTIME}(T^2)$ din lema, şi, deci $\mathcal{P} \subseteq \mathcal{NP}$. Este $\mathcal{P} = \mathcal{NP}$? Adică maşinile Turing nedeterministe sunt o variantă a maşinilor Turing în raport cu clasa limbajelor decise în timp polinomial?

Arborii care reprezintă mulţimea operaţiilor unei maşini Turing nedeterministe au multe noduri (configurații) toate de înălţime medie. Singurul mod în care o maşină Turing determinstă poate concura cu o maşină Turing nedeterministă relativ la numărul de configuraţii ce pot fi atinse este prin operaţii cu număr exponenţial de paşi.

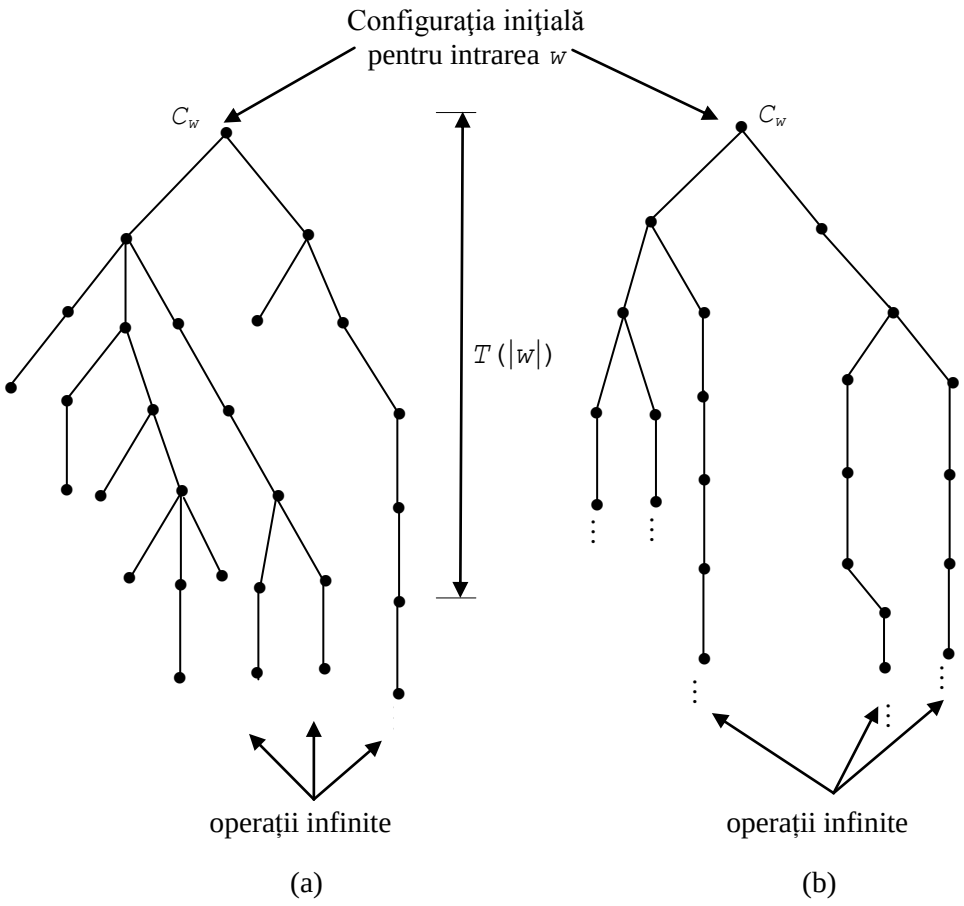


Figura 3-4

Definiţia 3.7

Funcția $f : \mathrm{N} \to \mathrm{N}$ este o funcţie de numărare a paşilor dacă există $k \geq 1$ şi o maşină Turing cu k benzi M astfel încât $\forall\ w \in \Sigma^*$, M se opreşte în $f(|w|)$ paşi pentru w,

$$(s, \#w\underline{\#}, \ldots, \underline{\#}) \vdash_M^{f(|w|)} (h, u_1\underline{a}_1v_1, \ldots, u_K\underline{a}_kv_k)$$

$u_1, \ldots, u_k, v_1, \ldots, v_k \in \Sigma^*$, $a_1, \ldots, a_k \in \Sigma$

Lema 3.3

Dacă p este un polinom cu coeficienţi nenegativi astfel încât

$\forall n \in \mathrm{N}, p(n) \geq 2n + 4,$

atunci p este o funcţie de numărare a pașilor unei mașini Turing.

Teorema 3.6

Fie L un limbaj acceptat în T_1, unde T_1 este o funcţie de numărare a pașilor de o mașină Turing $\mathrm{M}_1 = (K_1, \Sigma_1, \Delta_1, s_1)$. Atunci există numerele întregi $k_2 \geq 1$, $r > 1$ şi o mașină Turing deterministă cu k_2 benzi $\mathrm{M}_2 = (K_2, \Sigma_2, \Delta_2, s_2)$ care decide L în T_2, unde $T_2(n) = r^{T_1(n)}$.

Demonstraţie. Fie M mașina Turing care pentru intrarea w se opreşte după $\mathrm{T}_1(|w|)$ paşi în modul descris în Definiţia 3.7. Este garantată existența lui M pentru că T_1 este o funcţie de numărare a pașilor. Fie k_2 numărul de benzi ale lui M_2, $k_2 = k + 2$, unde k este numărul de benzi ale lui M.

M_2 copiază intrarea pe cea de a k_2-a banda; fie @ maşina care efectuează această funcţie. Apoi M_2 cedează controlul unei versiuni M' a lui M pentru intrarea w. M′ este identică cu M, cu excepţia unei bande suplimentare a (k_2−1)-a pe care scrie câte un simbol d_1 la fiecare pas al lui M.

La sfârşitul operaţiilor lui M′, banda suplimentară conţine şirul $d_1^{T_1(|w|)}$. M_2 îşi şterge benzile cu excepţia celei de a (k_2−1-a şi k_2-a. Fie $\mathcal{E}$ maşina Turing care realizează această funcţie. M_2 scrie $ pe prima bandă, se deplasează la dreapta şi intră într-un ciclu în care simulează determinist operaţiile lui M_1 pentru w. Acest ciclu constă în trei maşini, $\mathcal{G}$, M_1' şi $\mathcal{G}$'. Maşina $\mathcal{G}$ copiază w de pe banda k_2 pe prima bandă. M_1′ este o variantă a lui M_1 care simulează determinist M_1 pe prima bandă, consultând banda (k_2−1) pentru alegeri nedeterministe. Când M_1, s-ar fi oprit, M_1′ şterge benzile şi se opreşte cu #Ⓨ# pe prima bandă. Pentru a trece de la simularea unei operaţii a lui M_1 la alta, M_2 utilizează maşina $\mathcal{G}$' care calculează în baza r_1, unde r_1 este numărul maxim posibil al alegerilor

nedeterministe pentru un pas a lui M_1. Diferenţa dintre $\mathcal{G}$ şi $\mathcal{G}$' este că prima dată $\mathcal{G}$' încearcă să crească lungimea şirului, şterge toate benzile şi scrie Ⓝ pe prima bandă. M_2 este reprezentată în Figura 3-5.

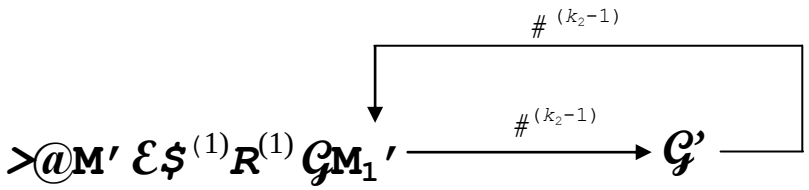


Figura 3-5

Numărul de paşi executaţi de M_2 pentru w, $|w| = n$, este:

1. @ necesită $3n+3$ paşi;
2. M' necesită $2T_1(n)$ paşi, conform definiţiei sale;
3. $\mathcal{E}$ necesită cel mult $4T_1(n)$ paşi
4. Fiecare dintre ciclurile lui $\mathcal{G}$, M_1', $\mathcal{G}$' (cel mult $r_1^{T_1(n)}$) necesită cel mult $4T_1(n)$ paşi.
5. $2T_1(n)+5$ paşi sunt necesari pentru a şterge toate benzile şi a scrie Ⓝ.

În total, $8T_1(n) + 3n + 8 + r_1^{T_1(n)} 4T_1(n)$ paşi. Ţinând cont că:

$$4T_1(n) \leq 2^{4T_1(n)}$$

$$2T_1(n) \geq 3n + 8$$

limita superioară este: $10T_1(n) + (16r_1)^{T_1(n)} \leq (16r_1 + 1)^{T_1(n)}$. Aceasta demonstrează teorema pentru $r = 16r_1+1$. ■

Corolar

$\mathcal{NP} \subseteq \cup \{\text{TIME}(r^{n^d}) : r, d > 0\}$

Definiţia 3.8

Fie $L \subseteq \Sigma^*\$\Sigma^*$ un limbaj, unde $\$ \notin \Sigma$. Spunem că L este balansat polinomial dacă există un polinom p astfel încât $x\$y \in L$, numai dacă $|y| \leq p(|x|)$.

În particular, dacă $L \subseteq \Sigma^*\$\Sigma^*$ şi $\$ \notin \Sigma$, atunci $L/\$\Sigma^*$ este mulţimea tuturor $x \in \Sigma^*$ astfel încât $x\$y \in L$, $y \in \Sigma^*$.

Teorema 3.7

Fie $L \subseteq \Sigma^*$ un limbaj, unde $\$ \notin \Sigma$, $|\Sigma| \geq 2$. Atunci $L \in \mathcal{NP}$ dacă şi numai dacă există un limbaj polinomial balansat $L' \subseteq \Sigma^*\$\Sigma^*$ astfel încât $L' \in \mathcal{P}$ şi $L = L'/\$\Sigma^*$.

Exemplu 3.4

Să formalizăm problema comis voiajorului. Datele sunt:

(a) n - numărul de oraşe care trebuie vizitate, identificate prin $1, 2, \ldots, n$.

(b) distanţele dintre oraşe, date printr-o matice D, de dimensiune $n \times n$, numită matricea distanţelor; intrarea d_{ij} din matrice reprezintă distanţa dintre oraşele i, j (nu se impune ca D să fie simetrică, adică $d_{ij} = d_{ji}$, pentru orice i, j); $d_{ii} = 0$, $\forall\, i$; vom reprezenta matricea printr-o codificare α.

Se cere să se găsească drumul cel mai scurt.

Un tur t poate fi definit ca o bijecţie $t : \{1, 2, \ldots, n\} \rightarrow \{1, 2, \ldots, n\}$. Dacă $t(i) = j$, atunci oraşul i va fi vizitat al j-lea în tur. Lungimea unui tur t este:

$$D(t) = \sum_{j=1}^{n-1} d_{t(j),\,t(j+1)} + d_{t(n),\,t(1)}$$

Determinarea turului t pentru care $D(t)$ este minim nu este o problemă de decizie de limbaj, este o problemă de evaluare de funcţie. Pentru a formula problema în termenii unui limbaj, presupunem că avem un întreg b, reprezentând un buget. Întrebarea este dacă există un tur astfel încât $D(t) \leq b$.

Se poate arăta că dacă avem un algoritm cu timp polinomial pentru a decide dacă există un tur cu lungime într-o limită dată, putem utiliza acel algoritm pentru a dezvolta un nou algoritm în timp polinomial pentru a calcula turul minim. Formal, definim limbajul

$$PCV = \{ I^n @ \alpha(D) @ I^b : \text{există } t \text{ astfel încât } D(t) \le b \}$$

Nu se poate afirma că $PCV \in \mathcal{P}$. Toţi algoritmii cunoscuţi pentru rezolvarea acestei probleme necesită un timp exponenţial.

Este, însă, uşor de arătat că $PCV \in \mathcal{NP}$, utilizând teorema anterioară. Este suficient să observăm că:

$$PCV' = \{ I^n @ \alpha(D) @ I^b \$ \alpha(t) : D(t) \le b \}$$

este polinomial balansat şi este în $\mathcal{P}$, unde prin $\alpha(t)$ am codificat turul t astfel încât

$$\alpha(t) = I^{t(1)} c I^{t(2)} c \ldots c I^{t(n)}$$

PCV' este balansat polinomial pentru că $|\alpha(t)| \le n^2$.

Date fiind t şi D se poate evalua $D(t)$ pentru a decide dacă $D(t) \le b$ în timp polinomial. Prin urmare $PCV \in \mathcal{NP}$.

Putem spune că $\mathcal{P}$ este clasa tuturor mulţimilor L astfel încât apartenenţa unui element x la L poate fi testată eficient.

3.4. $\mathcal{NP}$ Completitudine

Definiţia 3.9

Fie Σ, Δ alfabete. O funcţie $f : \Sigma^* \to \Delta^*$ este calculabilă în timpul T printr-o maşină Turing deterministă $M=(K, \Sigma', \delta, s)$ cu k benzi, dacă şi numai dacă, $\forall x \in \Sigma^*$,

$$(s, \#x\underline{\#}, \underline{\#}, \ldots, \underline{\#}) \vdash_M^t (h, \#f(x)\underline{\#}, \underline{\#}, \ldots, \underline{\#}), t \le T(|x|).$$

Spunem că f este calculabilă în timpul T dacă există maşina Turing M care calculează f în timpul T. Spunem că f este calculabilă polinomial dacă există un polinom T astfel încât f este calculabilă în T.

Rezultă imediat că dacă f este calculabilă în timpul T, atunci $\forall x \in \Sigma^*$, $|f(x)| \le T(|x|) + |x|$, pentru că maşina care calculează f nu poate produce în $T(|x|)$ ca ieşire $f(x)$ cu lungime mai mare decât $T(|x|) + |x|$.

Exemplu 3.5

Dacă L un limbaj din $\mathcal{P}$, atunci funcţia f_L:

$$f_L(x) = \begin{cases} \text{Ⓨ}, & x \in L \\ \text{Ⓝ}, & \text{altfel} \end{cases}$$

este, din definiţia lui $\mathcal{P}$, calculabilă polinomial.

Definiţia 3.10

Fie $L_1 \subseteq \Sigma_1^*$ şi $L_2 \subseteq \Sigma_2^*$ limbaje. O funcţie calculabilă polinomial τ: $\Sigma_1^* \to \Sigma_2^*$ este numită transformare în timp polinomial din L_1 în L_2, dacă şi numai dacă, $\forall\ x \in \Sigma_1^*$, $x \in L_1$ dacă şi numai dacă $\tau(x) \in L_2$.

Dacă există o transformare în timp polinomial a lui L_1 în L_2, este posibil să se adapteze o procedură de decizie pentru L_2 cu scopul de a obţine una pentru L_1.

Pentru a determina dacă x este în L_1, se poate calcula $\tau(x)$ şi se verifică dacă şirul este în L_2.

Lema 3.4

Dacă $\tau_1 : \Sigma_1^* \to \Sigma_2^*$ şi $\tau_2 : \Sigma_2^* \to \Sigma_3^*$ sunt transformări în timp polinomial din L_1 în L_2 şi din L_2 în L_3, atunci $\tau_1 \circ \tau_2 : \Sigma_1^* \to \Sigma_3^*$ este o transformare în timp polinomial din L_1 în L_3.

Demonstraţie. Presupunem că τ_1 este calculat de maşina Turing M_1 în timpul polinomial T_1 şi τ_2 este calculat de M_2 în T_1 (polinomial). Atunci $\tau_1 \circ \tau_2$ poate fi calculat de M_1M_2.

Pentru intrarea $x \in \Sigma_1^*$, M_1M_2 va calcula $\tau_1 \circ \tau_2(x)$ într-un timp mărginit de $T_1(|x|) + T_2(T_1(|x|) + |x|)$ care este polinomial.

Rămâne de arătat că $x \in L_1$ dacă şi numai dacă $\tau_1 \circ \tau_2(x) \in L$, ceea ce este trivial.

$x \in L_1 \Leftrightarrow \tau_1(x) \in L_2 \Leftrightarrow \tau_2(\tau_1(x)) = \tau_1 \circ \tau_2(x) \in L_3$ ■

Definiţia 3.11

Un limbaj $L \subseteq \Sigma^*$ este numit $\mathcal{NP}$ complet, dacă şi numai dacă:
(a) $L \in \mathcal{NP}$
(b) $\forall L' \in \mathcal{NP}$, există o transformare în timp polinomial de la L' la L.

Teorema 3.8

Fie L un limbaj $\mathcal{NP}$ complet. Atunci $\mathcal{P} = \mathcal{NP}$ dacă și numai dacă $L \in \mathcal{P}$.

Demonstraţie. Presupunem $\mathcal{P} = \mathcal{NP}$. Cum L este $\mathcal{NP}$ complet, din Definiţia 3.11, $L \in \mathcal{NP}$, prin urmare $L \in \mathcal{P}$.
Implicația inversă. Presupunem că L este decis de o mașină Turing deterministă M_1 în T_1 (polinomial), şi fie L' un limbaj în $\mathcal{NP}$; vom arăta că $L' \in \mathcal{P}$.
Cum L este $\mathcal{NP}$ complet şi $L' \in \mathcal{NP}$ atunci există o transformare τ în timp polinomial de la L' la L. Să presupunem că τ este calculat de o mașină Turing M_2 în T_2 (polinominal). Atunci afirmăm că mașina Turing M_2M_1 decide L' în timp polinomial.
Pentru a vedea dacă M_2M_1 decide L', trebuie observat că M_2M_1 se opreşte cu Ⓨ pe bandă pentru intrarea x dacă și numai dacă $\tau(x) \in L$; cum τ este o transformare în timp polinomial, $\tau(x) \in L$ dacă și numai dacă $x \in L'$. Mai mult, M_2M_1 se opreşte pentru intrarea x în timpul $T_2(|x|) + T_1(T_2(|x|)+|x|)$ care este polinomial în $|x|$.
Cum L' poate fi orice limbaj în $\mathcal{NP}$, concluzionăm că $L' \in \mathcal{P}$, şi din $\mathcal{NP} \subseteq \mathcal{P}$ rezultă că $\mathcal{P} = \mathcal{NP}$. ■

Limbajele $\mathcal{NP}$ complete joacă în raport cu intrebarea $\mathcal{P} = \mathcal{NP}$, acelaşi rol pe care limbajul K_0 îl joacă în raport cu întrebarea dacă toate limbajele acceptate Turing sunt decidabile Turing.

Dar cum găsim un limbaj $\mathcal{NP}$ complet? Din nou construcţia lui K_0 oferă soluţia. Cum K_0 constă din codificarea maşinii Turing şi a intrării acceptate, vom considera codificarea maşinii Turing nedeterministe, intrările acceptate şi numărul de paşi necesari pentru acceptarea acelor intrări. Fie

$N_0 = \{\rho(M)\rho(w)@I^t$: maşina Turing nedeterministă care acceptă w în cel mult t paşi$\}$.

Teorema 3.9

N_0 este $\mathcal{NP}$ complet.

Noţiunea de $\mathcal{NP}$ completitudine a eşuat în a demonstra $\mathcal{P} = \mathcal{NP}$. Totuşi $\mathcal{NP}$ completitudinea este o idee importantă, pentru că dintre toate limbajele $\mathcal{NP}$ – limbajele $\mathcal{NP}$ complete sunt cel mai puţin probabil de a fi decise în timp polinomial. Conceptul de $\mathcal{NP}$ completitudine a unificat experienţa cercetărilor în diverse domenii arătând că niciuna dintre aceste probleme nu poate fi rezolvată printr-un algoritm polinomial decât dacă $\mathcal{P} = \mathcal{NP}$.

4. MODELE ECHIVALENTE DE CALCULABILITATE

Mașinile Turing reprezintă versiunile formale ale algoritmilor şi nicio procedură nu poate fi considerată un algoritm dacă nu poate fi reprezentată printr-o mașină Turing (Teza lui Church). Este o teză, nu o teoremă, pentru că nu este un rezultat matematic, asociază un obiect matematic unui concept informal.

Sunt mașinile Turing (automatele în general) singura abordare a ideii de calculabilitate? Nu. Vom considera alte două alternative:

- generatoarele de limbaje – gramaticile nerestricționate
- funcţiile definite pe mulțimea numerelor naturale: funcții μ-recursive.

Vom arăta că mașinile Turing pot calcula funcţii μ-recursive, încheind astfel un ciclu de simulări: mașinile Turing poate fi simulate prin gramatici, care pot fi simulate prin funcții μ-recursive care pot fi simulate prin mașini Turing.

Singura concluzie este că toate aceste abordări ale ideii de calculabilitate sunt echivalente. Este teza lui Church extinsă la reprezentări diferite de automate.

4.1. Gramatici

Definiţia 4.1

O gramatică este un tuplu $G = (V, \Sigma, R, S)$, unde

V este un alfabet;

$\Sigma \subseteq V$ este mulţimea simbolurilor terminale; $V - \Sigma$ este numită mulțimea simbolurilor neterminale;

$S \in V - \Sigma$ este simbolul de start;

$R \subseteq (V^*(V-\Sigma)V^*) \times V^*$ este mulţimea regulilor (o submulţime finită);

Scriem $u \to v$, dacă $(u, v) \in R$. Notăm $u \underset{G}{\Rightarrow} v$ dacă şi numai dacă $w_1, w_2 \in V^*$ şi $u' \to v'$, $u = w_1 u' w_2$ şi $v = w_1 v' w_2$. Notaţia $\overset{*}{\underset{G}{\Rightarrow}}$ reprezintă închiderea reflexivă şi tranzitivă a relaţiei $\underset{G}{\Rightarrow}$

Un şir $w \in \Sigma^*$ este generat de gramatica G dacă şi numai dacă $S \overset{*}{\underset{G}{\Rightarrow}} w$. Limbajul generat de gramatică, $L(G)$, este mulţimea tuturor şirurilor din Σ^* generate de G.

O derivare este o secvenţă de forma

$$w_0 \underset{G}{\Rightarrow} w_1 \underset{G}{\Rightarrow} w_2 \underset{G}{\Rightarrow} \dots \underset{G}{\Rightarrow} w_n$$

Exemplu 4.1

Vrem să definim gramatica G care generează limbajul $L(G) = \{ w \in \{a, b, c\}^* \mid w$ are un număr egal de simboluri $a, b, c\}$.
Regulile gramaticii sunt următoarele :

$S \to e$

$S \to ABCS$ // generează şiruri de tipul $(abc)^n$, $n > 0$

Trebuie să introducem reguli pentru a permuta simbolurile A, B, C şi a înlocui apoi neterminalele A, B, C cu a, b, respectiv c :

$AB \to BA$

$AC \to CA$

$BC \to CB$

$BA \to AB$

$CA \to AC$

$CB \to BC$

$A \to a$

$B \to b$

$C \to c$

Să considerăm derivarea şirului *ccbaba*

$$\underline{S} \Rightarrow ABC\underline{S}$$
$$\Rightarrow ABCABC\underline{S}$$
$$\Rightarrow ABCA\underline{BC}$$
$$\Rightarrow ABC\underline{AC}B$$
$$\Rightarrow A\underline{BC}CAB$$
$$\Rightarrow \underline{AC}BCAB$$
$$\Rightarrow CA\underline{BC}AB$$
$$\Rightarrow C\underline{AC}BAB$$
$$\Rightarrow CC\underline{AB}AB$$
$$\Rightarrow CCBA\underline{AB}$$
$$\Rightarrow \underline{C}CBABA$$
$$\Rightarrow c\underline{C}BABA$$
$$\Rightarrow cc\underline{B}ABA$$
$$\Rightarrow ccb\underline{A}BA$$
$$\Rightarrow ccba\underline{B}A$$
$$\Rightarrow ccbab\underline{A}$$
$$\Rightarrow ccbaba$$

Exemplu 4.2

Să definim gramatica *G*, care generează limbajul:

$$L(G) = \{a^{2^n} \mid n \geq 0\}$$

$G = (V, \Sigma, R, S)$
$V = ([,], A, D, S, a)$
$\Sigma = \{a\}$
$R = \{$
$S \rightarrow [A]$
$[\rightarrow [D$
$D] \rightarrow]$

$DA \to AAD$

$[\to e$

$] \to e$

$A \to a$

}

Intuitiv, gramatica generează şirurile astfel: pornind cu simbolul de start S, şirul $[A]$ este obţinut aplicând regula $S \to [A]$; şirul $[D^nA]$ este produs prin $n \geq 0$ aplicări ale regulii $[\to [D$. Simbolurile D sunt pentru a dubla neterminalul A; de câte ori se trece peste un A de la stânga la dreapta, se înlocuieşte neterminalul A cu doi A folosind regula $DA \to AAD$, la sfârşit $[D^nA]$ devine $[A^{2^n}D^n]$. Simbolurile D sunt eliminate prin regula $D] \to]$, obţinând $[A^{2^n}]$; simbolurile [,] sunt eliminate prin regulile $[\to e$, respectiv $] \to e$. La sfârşit se aplică regula $A \to a$ pentru a obţine şirul de simboluri terminale.

Trebuie să arătăm că niciun şir din $\{a\}^*$ care nu este de forma a^{2^n} nu poate fi generat de G. Numărul simbolurilor D din şirurile intermediare este schimbat doar prin aplicarea regulilor $[\to [D$ şi $D] \to]$; simbolurile [şi] pot apărea doar o dată fiecare la stânga şi dreapta şirului. Dacă regula $] \to e$ este aplicată prematur (înainte eliminării simbolurilor D) nu pot fi produse şiruri de simboluri terminale. Deci singurul mod în care pot fi produse şiruri de simboluri terminale este să se genereze simboluri D la capătul stânga, apoi simbolurile A şi la sfârşit să se elimine simbolurile D de la capătul dreapta. Acest proces lasă şirul în forma $\{[, e\}\}A, a\}^{2^n}\{]\}$ unde n este numărul de simboluri D produse. Singurul şir de terminale ce poate fi generat este a^{2^n}.

Vom arăta că operaţiile unei maşini Turing pot fi simulate prin derivări ale unei gramatici. În acest scop vom reprezenta configuraţia maşinii Turing prin şiruri, apoi vom construi o gramatică care va deriva aceste şiruri unul din altul dacă şi numai dacă există relaţia $\vdash_M^*$ între configuraţiile maşinii Turing.

Lema 4.1

Fie $M = (K, \Sigma, \delta, s)$ o mașină Turing. Atunci există o gramatică G astfel încât pentru orice configuraţii (q, u, a, v) şi (q', u', a', v') ale lui M,

$$(q, u, a, v) \vdash_M^* (q', u', a', v')$$

dacă și numai dacă

$$[uqav] \overset{*}{\underset{G}{\Rightarrow}} [u'q'a'v']$$

Unde $[\ ,\ ]$ sunt simboluri care nu aparțin mulțimilor K sau Σ. Se presupune că mulțimile K, și Σ sunt disjuncte.

Demonstraţie. Ideea este să reprezentăm configuraţia (q, u, a, v) prin şirul $[uqav]$. Poziţia stării în şir reprezintă o indicaţie a poziţiei capului mașinii Turing, ceea ce face posibil ca partea stângă a unei reguli să conţină ca subşir o stare şi simbolul scanat. Cu excepţia detaliilor legate de mutarea pe o bandă vidă, doar starea, simbolul scanat şi poziţia capului trebuie schimbate la trecerea de la o configuraţie la alta. Simbourile $[\ ,\ ]$ apar la capetele reprezentării unei configurații. Simbolul $[$ este utilizat pentru a trata zona vidă a benzii, iar $]$ este inclus pentru legibilitate.

Fie $G = (V, \Sigma, R, S)$, unde

$$V = K \cup \Sigma \cup \{[,], h, S)\}$$

și R conţine regulile:

a) $\forall\ q \in K, a \in \Sigma, \delta(q, a) = (p, b), p \in K \cup \{h\}, b \in \Sigma$, atunci R conţine

$$qa \to pb$$

b) $\forall\ q \in K, a \in \Sigma$, dacă $\delta(q, a) = (p, R), p \in K \cup \{h\}, h \in \Sigma$, atunci R conţine

$$qab \to apb, \forall\ b \in \Sigma\ ,\text{ şi}$$

$$qa] \to ap\#]$$

c) $\forall\ q \in$ K, $a \in \Sigma$, $\delta(q, a) = (p, L), p \in K \cup \{h\}$, atunci R conţine

$$bqac \to pbac,$$

$\forall\ b \in \Sigma$, $c \in \Sigma \cup \{\,]\,\}$, astfel încât $a \neq \#$ sau $c \neq]$; şi

$bq\#] \rightarrow pb]$,

$\forall\ b \in \Sigma$, dacă $a = \#$.

Afirmăm că

$(q, u, a, v) \vdash_M^* (q', u', a', v')$ dacă şi numai dacă $[uqav] \overset{*}{\underset{G}{\Rightarrow}} [u'q'a'v']$

Dacă este adevărat atunci este adevărat şi pentru $\vdash_M^*$ şi $\overset{*}{\underset{G}{\Rightarrow}}$ Demonstraţia este imediată şi se bazează pe definiţia $\vdash_M$. ■

Exemplu 4.3

Fie $M = (\{q_0, q_1\}, \{a, \#\}, \delta, q_0)$ şi δ dată prin tabelul:

q	σ	$\delta(q, \sigma)$
q_0	a	(q_1, R)
q_0	$\#$	(q_0, a)
q_1	a	(q_0, L)
q_1	$\#$	(q_1, a)

Gramatica asociată $G = (V, \{a, \#\}, R, S)$, unde

$V = \{q_0, q_1, a, \#, [,], h, S\}$

şi R conţine regulile

(a) 1. $q_0\# \rightarrow q_0a$
2. $q_1\# \rightarrow q_1a$

(b) 1. $q_0aa \rightarrow aq_1a$
2. $q_0a\# \rightarrow aq_1\#$
3. $q_0a] \rightarrow aq_1\#]$

c) 1. $aq_1aa \rightarrow q_0aaa$
2. $\#q_1aa \rightarrow q_0\#aa$
3. $aq_1a\# \rightarrow q_0aa\#$
4. $\#q_1a\# \rightarrow q_0\#a\#$
5. $aq_1a] \rightarrow q_0aa]$
6. $\#q_1a] \rightarrow q_0\#a]$

Mașina M pornind din $(q_0, \underline{\#})$ execută următorii pași:

$$(q_0, \underline{\#}) \vdash_M (q_0, \underline{a}) \vdash_M (q_1, a\underline{\#}) \vdash_M (q_1, a\underline{a}) \vdash_M (q_0, \underline{aa})$$

În gramatică putem scrie următoarele derivări:

$$[q_0\#] \underset{G}{\Rightarrow} [q_0a] \underset{G}{\Rightarrow} [aq_1\#] \underset{G}{\Rightarrow} [aq_1a] \underset{G}{\Rightarrow} [q_0aa]$$

utilizând regulile (a_1), (b_3), (a_2), (c_5).

Definiţia 4.2

a) Fie Σ_0 şi Σ_1 alfabete care nu conţin $\#$, şi fie funcția $f : \Sigma_0^* \to \Sigma_1^*$. Atunci spunem că f este calculabilă gramatical dacă și numai dacă există $G = (V, \Sigma, R, S)$, unde $\Sigma_0, \Sigma_1 \subseteq \Sigma$ şi există $x, y, x', y' \in V^*$ astfel încât $\forall\, u \in \Sigma_0^*$, $v \in \Sigma_1^*$,

$$f(u) = v$$

dacă și numai dacă

$$xuy \overset{*}{\underset{G}{\Rightarrow}} x'vy'$$

Spunem că G calculează f.

b) Fie o funcție $f : \mathrm{N} \to \mathrm{N}$. f este calculabilă gramatical dacă și numai dacă $f' : \{\mathrm{I}^*\} \to \{\mathrm{I}\}^*$ este calculabilă gramatical, unde $f'(I^n) = I^{(fn)}$, $\forall n \in \mathrm{N}$.

Exemplu 4.4

Fie $\Sigma_0 = \{a, b\}$, $f : \Sigma_0^* \to \Sigma_0^*$, $f(w) = w^R$. Atunci f este calculată de gramatica (V, Σ_0, R, S), unde

$$V = \{a, b, A, B, *, S, [,]\}$$

şi R conține regulile :

$$1.\ [a \to [A,\ [b \to [B$$

$$2.\ Aa \to aA,\ Ab \to bA,\ Ba \to aB,\ Bb \to bB$$

$$3.\ A* \to *a \qquad B* \to *b$$

Fie x, y, x', y' şirurile $[,*]$, $[*$ și $]$. G calculează f astfel: pornește cu șirul $[w*]$, $w \in \Sigma_0^*$, și utilizează reguli de tipul 1 și 2 pentru a converti simbolurile termnale (a, b) de la capătul stânga al lui w în simbolurile netermnale (A, B) și le propagă la capătul dreapta. Când ajung la limita

dreapta sunt transformate înapoi în simboluri terminale. Simbolul * este utilizat pentru a le separa de partea stânga a șirului. Dacă un simbol a întâlnit * rămâne neschimbat,

$[abb^*] \Rightarrow [Abb^*]$ $R1$

$\Rightarrow [bAb^*]$ $R2$

$\Rightarrow [bbA^*]$ $R2$

$\Rightarrow [bbx^*]$ $R3$

$\Rightarrow [Bb^*a]$ $R1$

$\Rightarrow [bB^*a]$ $R2$

$\Rightarrow [BB^*a]$ $R1$

$\Rightarrow [B^*ba]$ $R3$

$\Rightarrow [^*bba]$ $R3$

Teorema 4.1

Orice funcţie calculabilă Turing, definită pe şiruri numere, este calculabilă gramatical.

Demonstraţie. Fie $f : \Sigma_0^* \rightarrow \Sigma_1^*$ o funcție calculabilă Turing. Din Definiţia 1.5 există o mașină Turing (K, Σ, δ, s) astfel încât $\forall u \in \Sigma_0^*$, $f(u) = n$ dacă și numai dacă

$(s, \#u, \#, e) \vdash_M^* (h, \#v, \#, e)$

Aplicând construcția din Lema 4.1 se obţine o gramatică G. Fie şirurile x, y, x', y' definite în felul următor:

$x = [\#$

$y = s\#]$

$x' = [\#$

$y' = h\#]$

atunci G, x, y, x', y' respect cerința.

Cum calculabilitatea (gramaticală, respectiv prin mașina Turing) funcțiilor definite pe numere este definită în termenii calculabilității funcțiilor definite pe șiruri, demostrația este valabilă și pentru cele dintâi. ■

4.2. Funcţii primitiv recursive

În cele ce urmează, vom utiliza funcţii definite pe $N^k \to N$, $k \geq 0$, numite funcţii de ordin k.

Definiţia 4.3

Funcţii iniţiale.

a) Definim funcţia de ordin 0, $\xi: N^0 \to N$ astfel încât $\xi() = 0$

b) Fie $k \geq 1$, $1 \leq i \leq k$, π_i^k este funcţia proiecţie i de ordinul k.

$\pi_i^k: N^k \to N$

$\pi_i^k (n_1, \ldots, n_k) = n_i, \ n_1, \ldots, n_k \in N$

Dacă notăm cu $\tilde{n} = n_1, \ldots, n_k$,

$\pi_i^k(\tilde{n}) = n_i, \forall \ \tilde{n} \in N^k$

c) Funcţia succesor $\sigma: N^k \to N$, $\sigma(n) = n + 1$, $n \in N$

d) O funcţie iniţială este ξ, σ sau una dintre π_i^k.

Definiţia 4.4

Compunere şi recursivitate primitivă.

a) Fie $l > 0$, $k \geq 0$, g o funcţie de ordinul l şi $h_1, \ldots, h_l$ funcţii de ordinul k. Fie f o funcţie de ordinul k astfel încât $\forall \ \tilde{n} \in N^k$, $f(\tilde{n}) = g(h_1(\tilde{n}), \ldots, h_l(\tilde{n}))$. Spunem că f este obţinută din funcţiile $g, h_1, \ldots, h_l$ prin compunere.

b) Fie $k \geq 0$, g o funcţie de ordinul k, h o funcţie de ordinul $(k + 2)$. Fie f o funcţie de ordinul $(k + 1)$ astfel încât $\forall \ \tilde{n} \in N^k$, $f(\tilde{n}, 0) = g(\tilde{n})$ şi $\forall \ \tilde{n} \in N^*$, $m \in N$, $f(\tilde{n}, m+1) = h(\tilde{n}, m, f(\tilde{n}, m))$.

Atunci spunem că f este obţinută din g şi h prin recursivitate primitivă.

Definiţia 4.5

Funcţii primitiv recursive.
O funcţie este primitiv recursivă dacă este o funcţie iniţială sau poate fi obţinută dintr-o funcţie iniţială printr-o secvenţă de operaţii de compunere şi recursivitate primitivă.
Altfel spus, funcţiile primitive recursive formează cea mai mică clasă de funcţii ce conţine funcţiile iniţiale şi este închisă în raport cu compunerea şi recursivitatea primitivă.

Exemplu 4.5

Funcţia $\sigma^2 : N \to N$, $\sigma^2(n) = n + 2$, $\forall\ n \in N$ este primitiv recursivă. Fie $k = l = 1$, $g = h_1 = \sigma$:

$$\sigma^2(n) = g(h_1(n_1)) = \sigma(\sigma(n))$$

Funcția σ este o funcţie iniţială, prin urmare este primitiv recursivă. Funcția σ^2 este obţinută din funcţii primitiv recursive prin compunere deci este primitiv recursivă.

Ce justifică ideea, că la nivel intuitiv, orice funcţie primitiv recursivă este calculabilă? Funcţiile iniţiale sunt calculabile. Dacă f este obţinută din $g, h_1, \ldots, h_l$ prin compunere şi funcțiile $g, h_1, \ldots, h_l$ sunt calculabile atunci $f(\tilde{n})$ poate fi evaluată calculând mai întâi $h_1(\tilde{n}), \ldots, h_l(\tilde{n})$; obţinem valorile $m_1, \ldots, m_l$ şi apoi calculăm $g(m_1, \ldots, m_l)$.

Similar, dacă f este obţinută din g şi h prin recursivitate primitivă iar g şi h sunt calculabile atunci $f(\tilde{n}, m)$ poate fi evaluată astfel:

- calculăm $f(\tilde{n}, 0)$, adică $g(\tilde{n})$; dacă $m = 0$, am terminat; altfel fie $g(\tilde{n}) = p_0$
- calculăm $f(\tilde{n}, 1)$, adică $h(\tilde{n}, 0, p_0)$; fie p_1 rezultatul obținut; dacă $m = 1$, am terminat; altfel continuăm calculul
- calculăm $f(\tilde{n}, 2)$, adică $h(\tilde{n}, 1, p_1)$ …

Definiţia 4.6

Funcţii constante.

$\forall\ k \geq 0, j > 0$ definim a j-a funcţie constantă de ordinul k

$K_j^k : \mathrm{N}^k \to \mathrm{N}$, $K_j^k(\tilde{n}) = j$, $\forall\ \tilde{n} \in \mathrm{N}$

Lema 4.2

Funcţiile constante sunt primitiv recursive.

Demonstraţie. Vom demonstra prin inducţie dupa k.

$k = 0$: K_0^0 este ξ, care este o funcţie iniţială;

$K_{j+1}^0() = \sigma(K_j^0())$, $\forall\ j \geq 0$,

astfel K_{j+1}^0 este obţinut din σ şi K_j^0 prin compunere.

$k > 0$: dacă pentru un anumit k, K_j^k este primitiv recursivă $\forall\ j \geq 0$, atunci $\forall\ j \geq 0$ funcţia K_j^{k+1} este primitiv recursivă.

$K_j^{k+1}(\tilde{n}, 0) = K_j^k(\tilde{n})$

$K_j^{k+1}(\tilde{n}, m + 1) = \Pi_{k+2}^{k+2}(\tilde{n}, m, K_j^{k+1}(\tilde{n}, m))$

adică K_j^{k+1} este obţinut din K_j^{k+1} şi Π_{k+2}^{k+2} prin recursivitate primitivă. ■

Definiţia 4.7

Fie g o funcţie de ordinul k şi f o funcţie de ordinul l, unde $k, l \geq 0$. Presupunem că există funcţiile $h_1, \ldots, h_k : \mathrm{N}^l \to \mathrm{N}$, fiecare fiind o funcţie de proiecţie de ordinul l sau o funcţie constantă de ordinul l astfel încât $\forall\ \tilde{n} \in \mathrm{N}^l$,

$f(\tilde{n}) = g(h_1(\tilde{n}), \ldots, h_k(\tilde{n}))$.

Astfel f este obţinută din g prin transformare explicită.

Cu alte cuvinte, funcţia f este obţinută din g prin transformare explicită dacă f este obţinută din g prin duplicare, permutare sau substituire a valorii constante pentru argumentele lui g.

Lema 4.3

Dacă g este primitiv recursivă, atunci orice funcţie obţinută din g prin transformare explicită este primitiv recursivă.

Demonstraţie. Demonstraţia este imediată pentru că proiecţia precum şi funcţiile constante sunt primitive recursive. ■

Definiţia 4.8

a) Un predicat este o relaţie pe numerele naturale, adică o submulţime a lui N^k, pentru $k \geq 0$ (un astfel de predicat, este un predicat de ordin k). Notăm $P\tilde{n}$ pentru a indica $\tilde{n} \in P$.
b) Funcţia caracteristică a unui predicat P de ordinul k, este o funcţie de ordin k, $f : N^k \to \{0, 1\}$, $\forall\, \tilde{n} \in N^k$

$$f(\tilde{n}) = \begin{cases} 1, & P\tilde{n} \\ 0, & not\ P\tilde{n} \end{cases}$$

c) Un predicat este primitiv recursiv dacă şi numai dacă funcţia sa caracteristică este primitiv recursivă.

Lema 4.4

Dacă Q este un predicat primitiv recursiv atunci orice predicat obţinut din Q prin transformare explicită este primitiv recursiv.

Definiţia 4.9

Fie $k, l \geq 0$. Dacă Q este un predicat de ordinul k şi R este un predicat de ordinul l atunci conjuncţia lor este un predicat S de ordin $(k+l)$ astfel încât

$S\tilde{n}\tilde{m}$ dacă şi numai dacă $Q\tilde{n} \wedge R\tilde{m}$, $\forall\, \tilde{n} \in N^k$, $\forall\, \tilde{m} \in N^l$.

Similar, disjuncţia este un predicat T de ordinul $k+l$, astfel încât

$T\tilde{n}\tilde{m}$ dacă şi numai dacă $Q\tilde{n} \vee R\tilde{m}$, $\forall\, \tilde{n} \in N^k$, $\forall\, \tilde{m} \in N^l$.

Negaţia Q este o relaţie P de ordinul k, astfel încât

$P\tilde{n}$ dacă şi numai dacă $not\ Q\tilde{n}$.

Lema 4.5

Dacă Q și R sunt predicate primitiv recursive atunci conjuncţia, disjuncţia, negaţia lor sunt primitiv recursive.

Demonstraţie. Fie g și h funcţiile caracteristice primitiv recursive ale lui Q, R; definim f_1, f_2, f_3, funcţiile caracteristice ale conjuncţiei $Q \wedge R$, disjuncţiei $Q \vee R$, și negației $\bar{Q}$ unde

$f_1(\tilde{n}, \tilde{m}) = g(\tilde{n}) \infty h(\tilde{n}, \tilde{m})$

$f_2(\tilde{n}, \tilde{m}) = \mathrm{sg}(g(\tilde{n}) + h(\tilde{n}, \tilde{m}))$

$f_3(\tilde{n}) = 1 - g(\tilde{n})$ ■

Definiţia 4.8

Fie $k \geq 0$. Dacă Q este un predicat de ordinul $k + 1$ atunci cuantificarea sa existenţială este un predicat R de ordinul $k + 1$, astfel încât $\forall\ \tilde{n} \in \mathrm{N}^k$ şi $m \in \mathrm{N}$, $R\tilde{n}m$ dacă și numai dacă $\exists\ i$, $0 \leq i \leq m$, $Q\tilde{n}i$ este adevărat. Vom scrie:

$R\tilde{n}\tilde{m}$ dacă și numai dacă $\exists\ i \leq m[Q\tilde{n}i]$

Similar, cuantificarea universală a predicatului Q ste un predicat S de ordinul $k + 1$, astfel încât $\forall\ \tilde{n} \in \mathrm{N}^k$ şi $m \in \mathrm{N}$, $S\tilde{n}m$ dacă și numai dacă $\forall\ i$, $0 \leq i \leq m$, $Q\tilde{n}i$ este adevărat. Vom scrie:

$S\tilde{n}\tilde{m}$ dacă și numai dacă $\forall\ i \leq m[Q\tilde{n}i]$

Lema 4.6

Dacă Q este un predicat primitiv recursiv atunci cuantificările sale existenţiale şi universale sunt primitiv recursive.

Demonstraţie. Fie g funcţia caracteristică a lui Q; f_1, f_2 funcţiile caracteristice ale cuantificatorilor universal şi existențial

$$f_1(\tilde{n}, m) = \prod_{i=0}^{m} g(\tilde{n}, i)$$

$$f_2(\tilde{n}, m) = 1 - \prod_{i=1} (1 - g(\tilde{n}, i))$$

■

4.3. Gödelizare

Pentru a demonstra că am construit clasa funcţiilor care includ toate funcţiile calculabile gramatical, respectiv toate funcţiile calculabile Turing trebuie să mai facem doi paşi,

- extindem clasa funcţiilor primitiv recursive
- transformăm funcţiile definite pe mulţimea numerelor naturale în funcţii pe şiruri ($f : \mathrm{N} \to \mathrm{N} \Rightarrow f : \Sigma_0^* \to \Sigma_1^*$)

Fie Δ un alfabet şi $\beta = |\Delta| + 1$. Fixăm o anumită ordine a simbolurilor în Δ astfel încât $\Delta = \{d_1, \ldots, d_{\beta-1}\}$. Orice şir din Δ^* poate fi văzut ca un întreg în baza β.

Definim $g_n : \Delta^* \to \mathrm{N}$. Dacă $w = d_{i1}, \ldots, d_{ik}$, $k \geq 0$, $1 \leq i_j \leq \beta - 1$, $j = 1, \ldots, k$,

$$g_n(w) = \beta^{k-1} \infty i_1 + \beta^{k-2} \infty i_2 + \ldots + \beta^1 \infty i_{k-1} + i_k.$$

$$g_n(e) = 0$$

Spunem că $g_n(w)$ este numărul Gödel al şirului w.

Observaţie. g_n reprezintă codificarea şirurilor în numere; un şir corespunde unui anumit număr (unul singur), dar g_n nu este o bijecţie: un număr (de exemplu β) nu corespunde niciunui şir. Introducem un nou simbol d_0 corespunzând cifrei 0; g_n poate fi extinsă pentru a deveni o bijecţie,

$$g_n : \underbrace{\{e\} \cup \Delta(\Delta \cup \{d_0\})^*}_{\mathcal{D}} \to \mathrm{N}$$

Inversa este acum bine definită:

$$g_n^{-1} : \mathrm{N} \to \mathcal{D}$$

4.4. Funcţii μ-recursive

Definiţia 4.9

Fie $k \geq 0$ şi g o funcţie de ordinul $(k + 1)$. Minimizarea nemărginită a lui g este o funcţie f de ordinul k, astfel încât $\forall \tilde{n} \in N^k$,

$$f(\tilde{n}) = \begin{cases} \text{cel mai mic } m \text{ astfel încât } g(\tilde{n}, m) = 0, \text{ dacă un astfel de } m \text{ există} \\ 0, \text{ altfel} \end{cases}$$

A doua clauză garantează că funcția f este definită peste tot, indiferent de g. Scriem

$f(\tilde{n}) = \mu m[g(\tilde{n}, m)]$ şi spunem că f este obţinut din g prin minimizare nemărginită.

În general, minimizarea nemărginită a unei funcţii primitiv recursive nu trebuie să fie primitiv recursivă. Motivul este acela că nu există nicio metodă pentru a determina dacă un m de tipul cerut există. Totuşi, dacă g are proprietatea că un astfel de m există pentru orice $\tilde{n}$, atunci f este calculabilă şi g este calculabilă. Dându-se $\tilde{n}$, trebuie să evaluăm $g(\tilde{n}, 0)$, $g(\tilde{n}, 1)$, …, până când găsim un m astfel încât $g(\tilde{n}, m)=0$. În acest caz f nu trebuie să fie, în general, primitiv recursivă.

Definiţia 4.10

O funcţie g de ordinul $(k+1)$ este regulată dacă şi numai dacă pentru orice $\tilde{n} \in N^k$, există m astfel încât $g(\tilde{n}, m) = 0$. O funcţie este μ-recursivă dacă şi numai dacă poate fi obţinută din funcţiile iniţiale ξ, π_i^k, σ, prin următoarele operaţii:

- compunere
- recursivitate primitivă
- aplicarea minimizării nemărginite funcţiilor regulate.

Astfel, orice funcţie primitiv recursivă este μ-recursivă. Vom arăta că orice funcţie calculabilă gramatical este μ-recursivă. Vom considera un alfabet Δ şi codificarea Gödel g_n fixate ($\$ \in \Delta$). Fie $\Sigma_0, \Sigma_1 \subseteq \Delta - \{\$\}$.

Definiţia 4.11

O funcţie $f : \Sigma_0^* \to \Sigma_1^*$ este μ-recursivă dacăşi numai dacă funcţia f: N $\to$ N este μ-recursivă, unde $\forall n \in$ N

$$f'(n) = \begin{cases} gn(f(gn^{-1}(n))), & \text{dacă } gn^{-1}(n) \in \Sigma_0^* \\ 0, & \text{dacă } gn^{-1}(n) \notin \Sigma_0^* \end{cases}$$

Cu alte cuvinte, o funcţie f definită pe şiruri este μ-recursivă dacă funcţia corespunzătoare f' definită pe numere Gödel este μ-recursivă. Dacă argumentul nu este un număr Gödel, se consideră că f' are valoarea 0.

Teorema 4.2

Orice funcţie calculabilă gramatical definită pe şiruri sau pe numere este μ-recursivă.

4.5. Calculabilitatea Turing a funcţiilor μ-recursive

Vrem să arătăm că orice funcţie μ-recursivă poate fi calculată de o maşină Turing. Considerăm următoarele maşini Turing:

- S_L – deplasare stânga – transformă $\#w\underline{\#}$ în $w\underline{\#}$ (w nu conţine simboluri $\#$)
- S_R – deplasare dreapta – transformă $\#w\underline{\#}$ în $\#\#w\underline{\#}$ (w nu conţine simboluri $\#$)
- E – maşina de ştergere – transformă $\#w\underline{\#}$ în $\underline{\#}$ (w poate fi vid dar nu conţine simboluri $\#$)

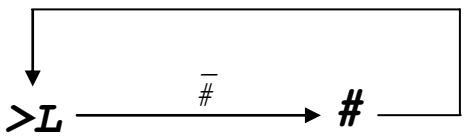


Figura 4-1

- *B* – maşina de backup – transformă $\#u\#w\underline{\#}$ în $\#w\#$; u, w pot fi vide, dar nu conţin simboluri $\#$

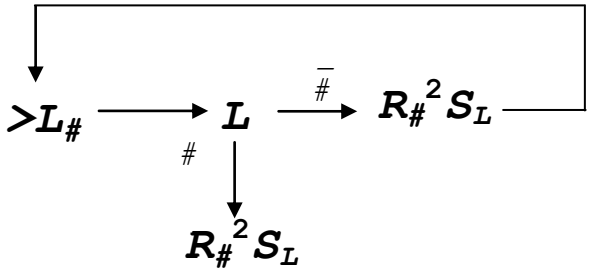


Figura 4-2

- C_k – maşina de copiere – produce o copie a şirului al k-lea de la stânga sa $(k \geq 1)$. Dacă şirurile w_1, …, w_k nu conţin simboluri $\#$, C_k transformă $\#w_1\#w_2\#\ldots\ \#w_k\underline{\#}$ în $\#w_1\#w_2\#\ldots\#w_k\#w_1\underline{\#}$

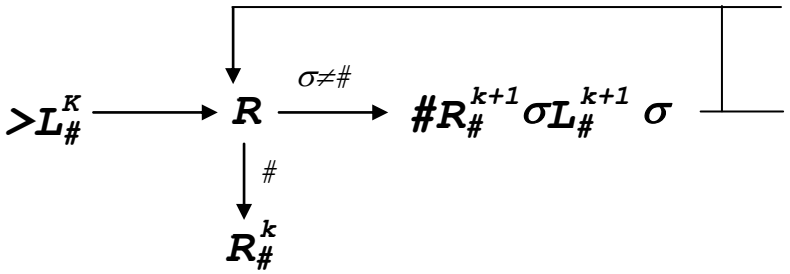


Figura 4-3

Observaţie. Maşina Turing C_k^l ($C_kC_k \ldots C_k$, l copii în total), $l \leq k$ are efectul de a copia şirurile k până la $k-l-1$ de la stânga poziţiei curente; transformă $\#w_1\#w_2\#\ldots\ \#w_k\underline{\#}$ în $\#w_1\#w_2\#\ldots\#w_k\#w_1\#w_2\#\ldots\#w_l\underline{\#}$

Teorema 4.3

Orice funcţie μ-recursivă definită pe numere sau pe şiruri este calculabilă Turing.

Demonstraţie. Vom începe prin a considera funcţii μ-recursive definite pe numere (cazul funcţiilor definite pe şiruri este prezentat printr-un corolar). Funcţiile μ-recursive sunt cele care pot fi obţinute din funcţiile iniţiale ξ, π_i^k, σ prin compunerea, recursivitatea primitivă şi minimizarea nemărginită aplicate funcţiilor regulate. Demonstraţia că orice funcţie μ-recursivă este calculabilă Turing, presupune demonstraţia că funcţiile iniţiale sunt calculabile Turing şi apoi demonstraţia că rezultatul compunerii, recursivităţii primitive şi a minimizării nemărginite aplicate funcţiilor calculabile Turing sunt calculabile Turing. Demonstraţia constă în demonstrarea mai multor leme.

Lema 1

Funcţiile iniţiale sunt calculabile Turing.

Demonstraţie.

a) Funcţia ξ *este* calculată de maşina Turing R care îşi mută capul o poziţie la dreapta şi se opreşte. Maşina porneşte din configuraţia iniţială $(s, \underline{\#})$ şi se opreşte în configuraţia finală $(h, \#\underline{\#})$, calculând funcţia ξ: $\mathrm{N}^0 \to \mathrm{N}^1$.

b) Funcţia de proiecţie π_i^k , $k \geq 1$, $1 \leq i \leq k$ este calculată de maşina Turing $E^{k-i}B^{i-1}$. E^{k-i} transformă $\#w_1\#w_2\#\ldots\#w_k\underline{\#}$ în $\#w_1\#w_2\#\ldots\#w_i\underline{\#}$, după care B^{i-1} transformă în $\#w_i\underline{\#}$. Maşinile E^0 $(k = i)$ şi B^0 $(i = 1)$ sunt considerate maşini triviale care se opresc imediat.

c) Funcţia succesor σ este calculată de maşina Turing IR care transformă $\#I^n\underline{\#}$ în $\#I^{n+1}\underline{\#}$, $(n \geq 0)$ ■

Lema 2

Fie $l > 0$, $k \geq 0$, g o funcţie calculabilă Turing de ordinul l, şi h_1, …, h_l funcţii calculabile Turing de ordinul k. Dacă f este obţinută din g, h_1, …, h_n prin compunere atunci f este calculabilă Turing.

Demonstraţie. Fie G, H_1 … H_l maşini Turing care calculează g, h_1 … h_l. Funcţia f, $f(\tilde{n}) = g(h_1(\tilde{n}), \ldots, h_l(\tilde{n}), \forall\ \tilde{n} \in \mathrm{N}$ este calculată de maşina Turing

$$C_k^k H_1 C_{k+1}^k H_2 \ldots C_{k+l-1}^k H_1 G B^k$$

$C_k^k H_1$ transformă

$\#w_1\#w_2\#...\#w_k\underline{\#}$ în

$w_1\#w_2\#...\#w_k\#w_1\#w_2\#...\#w_k\underline{\#}$ şi apoi în

$\#w_1\#w_2\#...\#w_k\#u_1\underline{\#}$, unde $w_i = I^{ni}$, $i = 1, \ldots, k$, $u_1 = I^{h1(\tilde{n})}$.
Similar, după $C_k^k H_1 C_{k+1}^k H_2 \ldots C_{k+1-1}^k H_1$ banda va fi

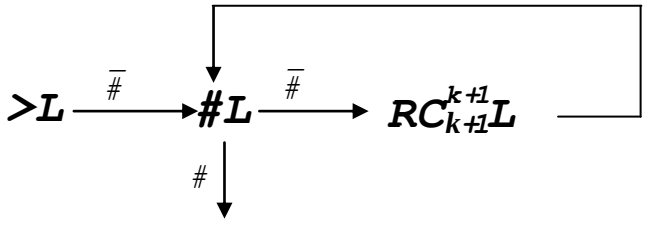
$\#w_1\#w_2\#...\#w_k\#u_1\#u_2\#...\#u_1\underline{\#}$
unde $u_i = I^{hi(\tilde{n})}$, $i = 1, \ldots, l$, care este transformat în

$\#w_1\#w_2\#...\#w_k\#v\underline{\#}$
de G şi apoi în $\#v\underline{\#}$ de B^k, unde $v = I^{f(\tilde{n})}$ ■

Lema 3

Fie $k \geq 0$, g o funcţie calculabilă Turing de ordinul k, şi h o funcţie calculabilă Turing de ordinul $k+2$. Dacă f este o funcţie obţinută din g şi h prin recursivitate primitivă atunci f este calculabilă Turing.

Demonstraţie. Fie G, H maşini Turing care calculează funcţiile g şi h. Construim o maşină Turing F care calculează f unde $\forall\ \tilde{n} \in N^k$, $f(\tilde{n}, 0) = g(\tilde{n})$ şi $\forall\ \tilde{n} \in N^k$, $m \in N^k$, $f(\tilde{n}, m+1) = h(\tilde{n}, m, f(\tilde{n}, m))$. F primeşte $\#w_1\#w_2\#...\#w_k\#u\underline{\#}$, unde pentru $n_1, \ldots, n_k, m \in N$, $w_i = I^{ni}$, $i = 1, \ldots, k$ şi $u = I^m$. Notăm $\overline{w} = w_1\#w_2\#...\#w_k$. F adaugă un simbol $ ca limită stânga a intrării, unde $ este simbol nou, neutilizat de G sau H. Intrarea este transformată în $\#\overline{w}\#u\#$, aplicând maşina Turing $F_1 = (S_R L_\#^2)^{k+1} \$ R_\#^{k+1}$. Conţinutul benzii devine $\#\overline{w}\#I^{m-1}\#\overline{w}\#I^{m-2}\# \ldots \#\overline{w}\#I^1\#\overline{w}\#I^0\#\overline{w}\underline{\#}$, adică $\# urmat de $\#\overline{w}\#I^i\#$, $i = m-1, m-2, \ldots, 0$, urmat de $\overline{w}\#$ cu capul la limita dreapta (dacă $m = 0$ atunci şirul este $\#\overline{w}\underline{\#}$; dacă $m = 1$ atunci şirul este $\#\overline{w}\#\#\overline{w}\underline{\#}$). Maşina Turing care relizează aceste transformări este maşina F_2.



$$RC_{k+1}^{k}$$

Figura 4-4

În final, mașinile Turing G şi H sunt utilizate pentru a calcula valori succesive $f(\tilde{n}, i)$, $i = 0, ..., m$. Notăm v_i pentru $I^{f(\tilde{n}, i)}$. Banda va arăta succesiv astfel:

$\$\#\overline{w}\#I^{m-1}\#\overline{w}\#I^{m-2}\#...\#\overline{w}\#I^{1}\#\overline{w}\#I^{0}\#v_0\underline{\#}$

$\$\#\overline{w}\#I^{m-1}\#\overline{w}\#I^{m-2}\#...\#\overline{w}\#I^{1}\#v_1\underline{\#}$

.
.
.

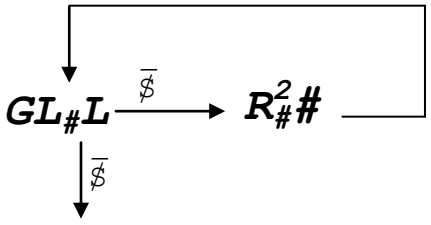
$\$\#\overline{w}\#I^{m-1}\#\overline{w}\#I^{m-2}\#...\#\overline{w}\#I^{i}\#v_i\#$

.
.
.

$\$\#\overline{w}\#I^{m-1}\#v_{m-1}\#$

$\$\#v\#$

La primul pas, $\overline{w}$ este eliminat de la capătul dreapta şi înlocuit prin şirul v_0 de mașina Turing G. La fiecare pas $\overline{w}\#I^{i}\#v_i$ este eliminat din dreapta şi înlocuit cu v_{i+1} de mașina Turing H. Înainte de inițierea fiecărui ciclu, simbolul aflat la stânga celui mai din dreapta şir de pe bandă este verificat; dacă nu este $ trebuie executat alt ciclu; dacă este $ operațiile sunt terminate, trebuie doar deplasat rezultatul la stânga pentru a elimina $. Această parte este executată de mașina Turing F_3.



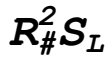


Figura 4-5

Maşina Turing care calculează f este $F_1F_2F_3$. ■

Lema 4

Fie g o funcţie regulată calculabilă Turing de ordinul $k+1$. Dacă f este o funcţie obţinută din g prin minimizare nemărginită atunci f este calculabilă Turing.

Demonstraţie. Fie G maşina Turing care calculează g. Construim o maşină Turing F care calculează f, unde $\forall\ \tilde{n} \in N^k$,

$f(\tilde{n})$ = cel mai mic m astfel încât $g(\tilde{n}, m)=0$.

Cum g este regulată atunci este garantată existenţa unui m pentru orice $\tilde{n}$. Maşina F primeşte intrarea în forma $\#w_1\#w_2\#...\#w_k\underline{\#}$, o transformă în $\#w_1\#w_2\#...\#w_k\#\underline{\#}$ (adică $\#w_1\#w_2\#...\#w_K\#I^0\#$), copiază întregul şir şi aplică G. Dacă rezultatul este diferit de 0 este şters şi un I este adăugat la dreapta şirului rămas pe bandă, care devine $\#w_1\#w_2\#...\#w_K\#I^1\underline{\#}$. Şirul este copiat şi maşina G este aplicată şirului $\#w_1\#w_2\#...\#w_k\#I^m\underline{\#}$. Maşina Turing B este utilizată pentru a elimina argumentele $w_1, ..., w_k$ şi lasă I^m pe bandă. Procesul se repetă până când se obţine 0. Specificarea lui F este următoarea.

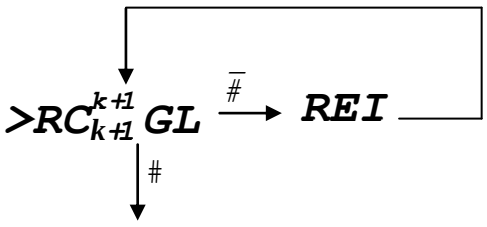




Figura 4-6

Lemele 1 - 4 demonstrează teorema pentru funcţii definite pe mulțimea numerelor naturale. Pentru a încheia demonstraţia, fie f o funcţie μ -recursivă definită pe şiruri. Atunci funcţia f' : N $\rightarrow$ N este μ -recursivă, unde pentru orice n astfel încât $gn^{-1}(n)$ este un şir în domeniul lui f, $f'(n) = gn(f(gn^{-1}(n)))$.
Prin argumentul dat f' este calculabilă Turing; presupunem că f' etse calculată de maşina Turing F'. Pentru a arăta că f este Turing-calculabilă putem proiecta o maşină Turing care primind un şir n din domeniului lui f.

- transformă $\#w\underline{\#}$ în $\#I^{gn(w)}\underline{\#}$ (converteşte şirul în reprezentarea unară a numărului Gödel asociat);
- utilizează F' pentru a transforma $\#I^{g(n)}\underline{\#}$ în $\#I^{f'(gn(w))}\underline{\#}$ adică în $I^{gn(f(w))}\underline{\#}$;
- transformă $\#I^{gn(f(w))}\underline{\#}$ în $\#f(w)\underline{\#}$, adică converteşte reprezentarea numărului Gödel a unui şir în şirul respectiv. ■

Singurele elemente care lipsesc sunt maşinile Turing care convertesc şiruri în/din reprezentări unare ale numărului Gödel (aceste maşini Turing sunt lăsate ca exerciţiu cititorului).

Demonstraţia că funcţiile calculabile Turing, calculabile gramatical şi μ-recursive sunt echivalente este completă. Aceste funcţii sunt numite funcţii recursive.

BIBLIOGRAFIE

1. **Brainerd, W. S., Landweber, H.**, *Theory of Computation*. New York: John Wiley, 1974.
2. **Church, A.**, *An Unsolvable Problem of Elementary Number theory*. American Journal of Mathematics, 58 (1936), 345-363.
3. **Hennie, F.,C.**, *Introduction to Computability*. Reading, Mass: Addison-Wesley, 1977.
4. **Hopcroft., J., E., Motwani, R., Ullman, J., D.**, *Introduction to Automata theory Languages, and Computation*. Addison-Wesley, 2001.
5. **Kleene, S., C.**, *General Recursive Functions of Natural Numbers*. Mathematische Annalen, 112 (1936), 727-742.
6. **Lewis, H., R., Papadimitriou, C., H.**, *Elements of the Theory of Computation*. N.J.: Prentice-Hall, 1998.
7. **Minsky, M., L.**, *Computation*: *Finite and Infinite Machines*. Englewood Cliffs, N.J.: Prentice-Hall, 1967.
8. **Papadimitriou, C., H., Steiglitz, K.**, *Combinatorial Optimization, Algorithms and complexity*. Mineola, New York: Dover Publications, Inc., 1982, 1998.
9. **Rogers Jr., H.**, *The Theory of Recursive Functions and Effective Computability*. New York: McGraw-Hill, 1967.

Nota: formalizarea folosită precum și exemplele prezentate se bazează pe cele prezentate în (4), și (6).